WAC
Windows Artefacts Collector

CENTRE D'EXCELLENCE CYBERDEFENSE AEROSPATIALE

Created by the Aerospace Cyber Defence Centre of Excellence (CEC)

Documentation
WAC (Windows Artefacts Collector)

Windows artefact collection software for digital forensic investigation



SYSTEM ARTEFACTS

USER FILES

DIGITAL EVIDENCE

A TOOL IN SUPPORT OF DIGITAL INVESTIGATION

WAC — Windows Artefacts Collector

User manual

Software version	3.0.0
Document edition	September 2026
Publisher	Aerospace Cyber Defence Centre of Excellence (CEC)
	French Air and Space Force Academy, Air Base 701, Salon-de-Provence

This document describes how to use WAC: preparing and running a collection, reading and verifying its results, and the traces it leaves on the examined machine. The figures quoted were measured on a Windows 11 test virtual machine; they are orders of magnitude, not commitments.

Contents

1 Overview **4**

1.1 What WAC is for 4

1.2 The guiding principle: a minimal footprint 4

1.3 What WAC does not do 4

2 How it works **5**

2.1 Reading the disk directly 5

2.2 Exhibit store and working copy 5

2.3 Registry hives and transaction logs 6

2.4 Two-pass extraction 6

2.5 What is still read live 6

3 Preparing a collection **7**

3.1 Requirements 7

3.2 The collection medium 7

3.3 Before going on site 8

4 Running a collection **9**

4.1 Command line 9

4.2 Configuration file: `wac.yml` 9

4.3 Options 13

4.4 Examples 14

4.5 The `--binary` option 15

4.6 What the screen shows 16

4.7 Duration 17

5 Collect here, convert elsewhere **18**

5.1 Why separate 18

5.2 The collection's settings 19

5.3 The two commands 19

5.4 What `--collect` takes 19

5.5 What `--convert` does . . . 20
5.6 Limitations . . . 20

6 The results **22**
6.1 Layout . . . 22
6.2 The exhibit store manifest . . . 22
6.2.1 Header . . . 22
6.2.2 Each exhibit . . . 23
6.3 Verifying a collection . . . 24
6.3.1 The seal . . . 24
6.3.2 Each exhibit . . . 24
6.4 The investigation log . . . 31
6.5 The artefact files . . . 31
6.5.1 System and accounts . . . 31
6.5.2 Program execution . . . 33
6.5.3 Files and browsing . . . 43
6.5.4 Devices . . . 46
6.5.5 Event logs (`--events`) . . . 46

7 Reading the data **48**
7.1 Timestamps . . . 48
7.2 Account names . . . 48
7.3 GUID names . . . 49
7.4 A missing field is not an empty field . . . 49
7.5 Artefact not collected . . . 49
7.6 Artefact absent from the version of Windows . . . 49
7.7 Coverage of MITRE ATT&CK . . . 60
7.8 Source expected but missing . . . 76
7.9 Undecoded data . . . 76
7.10 Unsupported format . . . 77
7.11 Paths . . . 78

8 Traces left on the examined machine **79**
8.1 What WAC still does . . . 79
8.2 What WAC no longer does . . . 80

8.3 What no tool can avoid 80

9 Known limitations **81**

10 Troubleshooting **82**

A Glossary **83**

CHAPTER 1

Overview

1.1 What WAC is for

WAC collects, from a running Windows computer, the artefacts a digital investigation needs: registry, event logs, Prefetch files, shortcuts and jump lists, scheduled tasks, services, accounts, processes and sessions. It delivers them as **JSON** files, ready to be analysed or fed into a processing pipeline.

It runs from a USB stick, without installation, and writes everything it produces to that stick.

1.2 The guiding principle: a minimal footprint

Examining a machine changes it. Every tool leaves traces: opened files whose last-access date changes, services that write to the logs when solicited, registry keys read through the API. These traces mix with the ones being looked for.

WAC is designed so that what it leaves behind is as close to nothing as the task allows, and so that **whatever it cannot avoid is written down**. In practice:

- the disk is read **directly**, read-only, without the system opening any file: no access date is modified;
- no key of the machine's registry is opened: the hives are copied, then read on the copy;
- no Volume Shadow Copy, no COM or WMI call, no logging service is solicited;
- nothing is written to the examined disk;
- every operation is recorded, with the trace it is expected to leave, in `investigation.json`.

1.3 What WAC does not do

- WAC **does not analyse**: it collects and formats. Interpretation is the analyst's.
- It does not make a full disk image: it extracts the files that hold artefacts.
- It does not capture memory. The only volatile data collected are the process list, the open sessions and the state of the services.

CHAPTER 2

How it works

2.1 Reading the disk directly

WAC opens the volume (`\\.\C:`) read-only, walks the NTFS file table (`$MFT`) itself and extracts the content of the files it needs. The system sees no file being opened: no access date updated, no directory walked, no lock.

This reading handles the NTFS features Windows normally hides:

NTFS compression
Windows 11 compresses, among others, the event log folder. WAC decompresses it itself.

Compact OS (WOF)
The system binaries of Windows 10 and 11 are stored compressed in a hidden stream; seen through the API, they look ordinary. WAC decompresses them (XPRESS variants).

Split attributes
A heavily fragmented file has its information spread over several records; WAC gathers them.

Preallocated files
Beyond the valid data length, NTFS returns zeros, even if the disk still holds remnants of former files. WAC does the same: the extracted content is the one any ordinary acquisition would see, and so is its fingerprint.

2.2 Exhibit store and working copy

A digital exhibit is never examined on itself: a copy is taken and sealed, and the work is done on a second copy. If the analysis damages something, the sealed copy remains and the operation can be redone.

WAC applies this procedure to **every** extracted file, including those it does not modify. The two directories keep their French names in the output:

`exhibits/` — the exhibit store
the raw copies, as read from the volume. Once copied to the working copy, this directory is touched no more, neither written nor read: only the check of its integrity, before a conversion copies it, reads it again. It holds the manifest identifying each exhibit, and the manifest's seal.

`working/` — the working copy
copied from the exhibit store and verified by fingerprint — all of it, `$MFT`, change journal and collected binaries included, which doubles their room on the medium. The only

modifications (replay of the hives' transaction logs) are made here, and every collector reads from here.

> **Keep in mind**
> Every extracted file is therefore written **twice** to the collection medium. Plan for the space (see section 3.2).

2.3 Registry hives and transaction logs

A hive copied from a running machine lacks its latest changes: they are still in its transaction logs (`.LOG1`, `.LOG2`). WAC extracts these logs and **replays them onto the working copy**, so that the analysis bears on the machine's actual state.

Everything WAC writes onto a copy can be undone: before the replay, the original content of every replaced page is saved in an undo journal (`<hive>.undo`). The raw copy stays intact in the exhibit store.

2.4 Two-pass extraction

Where each user's hives live is recorded in the `SOFTWARE` hive. WAC therefore first extracts the machine's hives (`SYSTEM`, `SOFTWARE`, `SAM`, `Amcache`), reads the profile list in the copy of `SOFTWARE`, then extracts each profile's hives (`ntuser.dat`, `usrClass.dat`).

Nothing is assumed about `C:`: the system drive is detected, and a profile on another disk (for instance `D:\Users\...`) is read on its own volume.

2.5 What is still read live

A few pieces of information describe the very moment of the collection and are held in no file. They are read live, with a single query each, and recorded as read in the exhibit store (`exhibits\live`) and at once in the working copy, where the conversion reads them back:

- the running **processes** (a kernel snapshot);
- the open **sessions**;
- the **current state of the services** (their configuration is read from the copied hive);
- the **clock** and the boot time;
- the running machine's **settings**: system drive, code page, time zone, volumes and their serial numbers.

CHAPTER 3

Preparing a collection

3.1 Requirements

- Windows 10 or later, 64-bit. WAC checks it at start-up.
- **Administrator rights**: reading a volume directly requires them. WAC checks this too.
- **NTFS**, **FAT** or **exFAT** volumes. A file on another file system (CD-ROM) cannot be read directly; it is reported as unread, never opened another way.

3.2 The collection medium

WAC runs from the collection medium (usually a USB stick) and writes all its results there. Choose an **empty** medium, preferably USB 3: writing is the most expensive part of a collection.

The space needed depends on the mode and the options. Two places fill up: the **exhibit store** (sealed raw copies) and the **working copy**, which takes only what the collectors modify or read during the collection (hives, Prefetch, shortcuts, jump lists, referenced binaries). The binaries acquired by `--collect --binary` and the NTFS metadata are never copied again: the conversion reads them in the exhibit store itself. Measured on the test virtual machine (Windows 11 without added software, 1 volume):

Collection	Exhibit store	Working copy	Time
`--collect --events --binary` (every executable)	2.7 GB	0.3 GB	8 min 30

The heaviest items of the exhibit store:

Content	Exhibit store
NTFS metadata: `$MFT` (380 MB), `$LogFile` (64 MB), `$UsnJrnl:$J` (35 MB, out of 3.6 GB logical: NTFS freed the start, which is not copied)	480 MB
Binaries copied: 2,853 not authenticated out of 44,446 read	1.2 GB
Event logs (`--events`)	170 MB
Hives, Prefetch, shortcuts, jump lists	180 to 230 MB

A machine used for a long time, or loaded with software, exceeds these figures: the `$MFT` grows with the number of files, the unauthenticated third-party binaries with the installed software. Allow **at least 5 GB free** for a collection only with `--binary`.

WAC checks the location **before its very first write** and refuses to start:

- if the working directory already holds files — an earlier collection would be analysed instead of this one;
- if the free space is clearly insufficient.

If space runs out while binaries are being collected, they are hashed without being copied, and the investigation log says so.

3.3 Before going on site

- Use a **new output folder** for each machine (option `--output`).
- Note the time of the intervention and the operator's identity: WAC records them too, but an independent record is useful.
- Keep in mind that plugging the stick in and running WAC leave unavoidable traces (chapter 8): they are known and documented, so that they are not mistaken for the activity under study.

CHAPTER 4

Running a collection

4.1 Command line

Open a command prompt **as administrator**, move to the collection medium, then run:

```
WAC.exe [--collect | --convert=folder] [--events] [--binary | --binary-all]
        [--threads=N] [--dump]
        [--output=folder] [--loglevel=N] [--debug]
WAC.exe --update-trust[=folder]
```

Every option is optional and disabled by default.

4.2 Configuration file: `wac.yml`

Version. WAC numbers its versions `<major>.<minor>.<bugfix>`: major for a rework, minor for features added, bugfix for fixes. The version is read in `WAC.exe`'s properties (Details tab), shown at start, and recorded in `investigation.json` (`Tool.Version`): a collection says which version made it. This document describes version 3.0.0.

The last build is in `bin` of the repository: `WAC.exe`, the reference `wac.yml`, the scripts that verify a collection (section 6.3) and `SHA256SUMS` to check them all. Copying `WAC.exe` and `wac.yml` to the collection key is enough; the scripts go with the investigator.

Rather than typing the options at every collection, the procedure is written once in `wac.yml`, next to `WAC.exe`: WAC applies it first, and an option given on the command line overrides it. `--write-config` writes the reference file, commented — the recommended procedure: unverified binaries collected, event logs included.

```
wac_version: 3.0.0      # version of WAC the file is written for (--write-config)
binary: unverified      # none | unverified | all
threads: 0              # analysis threads for the binaries, 0: automatic
output: output
log_level: 0
dump: false
language: fr            # fr | en: language of the console; the files stay in English
artefacts:              # one key per artefact; a missing key is true
  activities_cache:        true   # Windows Timeline: applications, files, pages,
    clipboard (ActivitiesCache.db)
  amcache:                 true   # Amcache.hve: applications, executables and drivers
    seen
  bam:                     true   # last execution of each program per user (Background
    Activity Moderator)
  bits:                    true   # qmgr.db: BITS jobs — URLs, destinations, notification
```

```
    command lines
  boot_records:            true   # what runs before Windows: each disk's first MiB (MBR,
    GPT), each partition's boot sectors, the EFI system partition's files
  cloud_identity:          true   # Entra ID join of the machine and work accounts of its
    users; token caches copied still encrypted
  defender:                true   # Microsoft Defender: support logs (MPLog, raw WPP
    traces), detection history, quarantine (copied encrypted)
  defender_configuration:  true   # Microsoft Defender exclusions and protection settings,
     its own and by policy
  defender_endpoint:       true   # Defender for Endpoint's local folder, where it is
    onboarded (Live Response outputs, cyber data), raw copy only
  deleted_files:                  # deleted files whose content the raw reading still
    finds
    enabled:               true   # false: nothing read
    level:                 2      # 1: the content of the $MFT's deleted records; 2: and
    the names left in the indexes
                                  # 3: and files carved from the free space (PE, ZIP, PDF)
     — the whole free space read
    max_file_mib:          64     # beyond: listed with its fingerprints, not copied
    max_total_mib:         4096   # all copied; the most recent first
    excluded_folders:             # never read, on every volume
      - \Windows\WinSxS
      - \Windows\SoftwareDistribution
      - \Windows\servicing
      - \Windows\Installer\$PatchCache$
      - \Windows\System32\catroot
      - \Windows\System32\catroot2
      - \$Extend\$RmMetadata
  devices:                 true   # USB and SCSI devices with their dates, portable
    devices, volume labels per drive letter, device interfaces
  dpapi_master_keys:       true   # DPAPI master keys of each user and of the system,
    copied still encrypted; JSON: owner, GUID, dates, never the key
  emails:                  true   # Outlook (PST, OST) and Thunderbird (mbox): messages,
    recipients, times; bodies and attachments exported
  error_reports:           true   # Windows Error Reporting: programs that crashed, with
    their path
  etw_logs:                true   # ETW trace files Windows keeps on its disk (.etl: boot
    sessions, diagnostics, Windows Update), raw copy only
  event_log_configuration: true   # each event log and boot ETW session: enabled, size,
    retention, access (a disabled one is a trace)
  events:                  true   # the event logs, with their messages
  explorer_recent_docs:    true   # files each user opened, by extension, most recent
    first (Explorer's RecentDocs)
  fat_volumes:             true   # FAT and exFAT volumes, a suspect's USB keys included:
    every entry, deleted ones too
  firewall:                true   # firewall rules and profiles
  firewall_log:            true   # the firewall's log (pfirewall.log), when logging is on
    : connections allowed or dropped, addresses, ports, process
  hosts:                   true   # the hosts file: names diverted to an address
  installed_programs:      true   # installed programs (Uninstall) of the machine and of
    each user
  internet_settings:       true   # each user's proxy and automatic configuration script
  intune_logs:             true   # Intune Management Extension's logs: scripts and
```

```
    applications the tenant pushed and ran, their outcome
  jump_lists:              true   # jump lists: files and places each user opened, per
    application
  known_networks:          true   # networks the machine joined: name, kind, gateway,
    first and last connection (NetworkList)
  local_groups:            true   # members of each local group (SAM)
  log_file:                true   # $LogFile and $TxfLog: NTFS and transactional NTFS
    journals, raw copy only
  mft:                     true   # $MFT: every file, present or deleted, with its dates (
    mft.json)
  mounted_devices:         true   # volumes and drive letters the machine knew (
    MountedDevices)
  mount_points:            true   # volumes and network shares each user's Explorer saw,
    with their commands (MountPoints2)
  mru:                     true   # files each user opened or saved through the common
    dialogs
  mru_apps:                true   # programs used to open or save those files
  muicache:                true   # names of the programs each user launched
  network:                 true   # live: connections, neighbours, interfaces, routes
  network_configuration:   true   # network adapters and their last IPv4/IPv6 settings and
     DHCP leases, read offline
  network_shares:          true   # folders the machine shares, with their path and
    permissions (LanmanServer)
  ntfs_volumes:            true   # $Boot and $UsnJrnl:$Max: volume serial and geometry,
    creation of the change journal
  office_trust_records:    true   # Office trusted documents: macros enabled by the user,
    and when
  onedrive:                true   # OneDrive's logs (.odl) and settings of each profile,
    its sync database included, raw copy only
  operating_system:        true   # version, installation, computer name, uptime
  persistence:             true   # registry persistence beyond Run: IFEO, AppInit_DLLs,
    Winlogon, LSA packages, COM …hijacking
  powershell_history:      true   # PSReadLine: the commands typed in PowerShell, per
    profile
  powershell_profiles:     true   # PowerShell profiles of the machine and of each user:
    scripts run at every start, a persistence
  prefetch:                true   # Prefetch: programs run, count, last eight runs, files
    loaded
  processes:               true   # live
  program_usage:           true   # programs each user launched or used: FeatureUsage,
    JumplistData, compatibility store and layers, RADAR, pinned items
  rdp_client:              true   # outgoing RDP connections: servers reached, .rdp files,
     bitmap cache tiles
  rdp_server:              true   # incoming RDP connections: authentications, sessions,
    sources (event logs)
  recent_documents:        true   # Recent: shortcuts to the files each user opened
  recycle_bin:             true   # $I of every volume's recycle bin: what was deleted, by
     whom, when
  registry_backups:        true   # RegBack: the machine's older hives, raw copy only
  root_certificates:       true   # root certificates of the machine and of each user
  run_keys:                true   # Run and RunOnce keys of the machine and of each user
  scheduled_tasks:         true   # scheduled tasks: commands, triggers, authors, dates
  search_index:            true   # Windows Search index: path, dates, size, owner,
```

```
  summary of each indexed item, deleted ones included
security:               true   # SECURITY hive: LSA secrets' names and dates, cached
  logons, audit policy (never a secret)
security_descriptors:   true   # $Secure:$SDS: owner and access rights of every file,
  each descriptor once (mft.json: SecurityId)
services:               true   # services and drivers
sessions:               true   # live
setupapi:               true   # setupapi.dev.log: device installations, first
  connection of a USB key
shadow_copies:          true   # shadow copies, read raw without the VSS service: hives
   and event logs as each copy kept them
shellbags:              true   # folders each user browsed in Explorer
shimcache:              true   # application compatibility cache (AppCompatCache)
shortcuts:              true   # shortcuts outside Recent: Start menu, desktops, Quick
  Launch, pinned task bar
srum:                   true   # SRUDB.dat: bytes sent and received, resources used,
  per application and user, 30 to 60 days
startup_folders:        true   # startup folders: what runs at logon
sysmon:                 true   # Sysmon where it runs: its configuration (rules as
  stored) and the deleted files it archived; its log is in events
thumbcache:             true   # Explorer's thumbnail and icon caches: what each user
  saw, even deleted since; images exported
typed_paths:            true   # paths typed in Explorer's address bar
usbstor:                true   # USB mass storage devices (Enum\USBSTOR)
userassist:             true   # programs each user launched from Explorer, with count
  and date
users:                  true   # local accounts (SAM): logons, failures, flags,
  password hashes
user_shell_folders:     true   # where each user's desktop, documents, downloads really
   are (OneDrive, another disk, a share)
usn_journal:            true   # $UsnJrnl:$J: creations, renames, deletions (
  usn_journal.json)
web_client:             true   # browsers: visits, downloads, searches (Edge, Chrome,
  Brave, Vivaldi, Opera); their databases copied raw
wmi:                    true   # WMI repository: event subscriptions (filters,
  consumers, bindings), read offline
word_wheel_query:       true   # terms searched in Explorer
zone_identifiers:       true   # Zone.Identifier streams (Mark of the Web): where
  downloaded files came from
```

- **Strict**: an unknown key, a value outside the ones allowed (`true`/`false` only, not `yes`), a duplicate key, a list: the collection is refused before it starts, with the line at fault. A key left out keeps its default: `true`.
- **Versioned**: `wac_version` carries the version of WAC the file was written for (`--write-config` puts it there). A file of another version, or without one, is refused before anything is read: it may lack settings added since, or carry some that are gone. Write it again with `--write-config`.
- **One key per artefact**, none collected unasked. The hives, the event logs and the `$MFT` are not artefacts but *dependencies*: taken when an artefact asked for reads them, and only then. The logs serve `events` and `rdp_server`; the users' hives `muicache`, `userassist`, `run_keys`, `shellbags`, `mru`, `mru_apps` and the other artefacts read in `NTUSER.DAT`; `Amcache`.

`hve` only `amcache`. SYSTEM, SOFTWARE, SAM, SECURITY and DEFAULT are always taken: they give the time zone, the profiles and the accounts' names.

- **Recorded**: the file's path, SHA-256 and content go into `investigation.json` (`Tool.Configuration`) — the procedure applied is part of the evidence.
- An artefact switched off is not read at all, and its JSON file says so (`CollectionStatus NotRequested`, with the key): never mistaken for a failure or an absence of traces.
- `--config=file` reads another file.
- Without `wac.yml`, WAC collects every artefact, event logs included, options as given.

4.3 Options

Option	Effect
`--collect`	The default mode, said explicitly: collection, on the examined machine — live observations and raw extraction into the sealed exhibit store, nothing converted. See chapter 5.
`--convert=folder`	Conversion only, on an analysis workstation, of the collection made with `--collect` in that folder, after checking the seal and every fingerprint. See chapter 5.
`--events`	Takes the event logs (`.evtx`, or `.evt` on Windows XP and Server 2003) even if `wac.yml` says `events: false`; otherwise they are taken by default (key `events`). Each event gets its readable message, rebuilt from its provider's resources. About 250 MB of copies.
`--binary`	At collection, reads directly every executable, library, driver, script and macro document of the fixed volumes, checks its signature in memory and **collects** into the exhibit store those it does not authenticate; at conversion, every file an artefact cites gets its exhibit's fingerprints. See section 4.5.
`--binary-all`	Like `--binary`, but **every** executable is collected, authenticated or not; the signature is checked all the same, and the manifest says for each whether it holds. For the cases where the content itself must be in the exhibit store.
`--threads=N`	Number of threads analysing the executables of `--collect --binary` (PE, Authenticode, catalogs, fingerprints). By default, the processor's threads minus one, the disk reading keeping its own. The result does not depend on it: the manifest is the same.

Option	Effect
`--update-trust [=folder]`	On the **analysis workstation**, **before** the collection, and alone: downloads from Microsoft the list of trusted roots (`authroot.stl`), that of disallowed certificates (`disallowedcert.stl`) and every root, checks their signatures and fingerprints, together with the vulnerable driver lists of Microsoft and LOLDrivers (not signed: authenticated by HTTPS alone) and the revocation lists (CRL) of the code signing authorities the CCADB lists, of Microsoft's own, and of those an earlier collection found missing (`wanted-crls.txt`), and writes them into `folder` (by default `trust`, next to `WAC.exe`), to be carried on the key with `WAC.exe`. The only mode that uses the network: the network library is loaded there alone. Refused with any other option. On Windows 7, the sites served over TLS 1.2 only (driver lists, CCADB) need update KB3140245: without it, those lists are missing from the trust set, and `investigation.json` says so.
`--dump`	Adds the raw hexadecimal value of shellbag and `.lnk` shortcut items, for later decoding.
`--output=folder`	Output folder, relative to the current directory: a name, without `\`, `/` nor `:`, other than `..` — the collection never leaves the key. Default: `output`.
`--language=fr|en`	Language of the console: steps, help, error messages. French by default; `language:` in `wac.yml` sets it too. The JSON files, `investigation.json` and the text log stay in English whatever the language: they are evidence read by tools, and a collection gives the same files whoever the operator.
`--loglevel=N`	Enables a detailed text log in the current directory, named after the program run (`WAC.exe.log` for `WAC.exe`): 0 none, 1 per artefact type, 2 per artefact, 3 per sub-function (debugging).
`--debug`	Prints the NTFS reader's trace on standard error (path resolution, index blocks, cluster runs). For diagnosis only.
`--help, /?`	Shows the help.

4.4 Examples

Standard collection:

```
E:\> WAC.exe --output=accounting-pc-01
```

Full collection, with logs, binaries and a detailed log:

```
E:\> WAC.exe --events --binary --output=accounting-pc-01 --loglevel=2
```

4.5 The `--binary` option

A fingerprint is enough to query a public database (VirusTotal, for instance) **without sending it the file**. But it says nothing about a binary nobody knows — precisely the one that matters to the investigation — and a binary left behind may be gone by the time a detection comes in. This is why `--binary` also copies potential payloads:

- **collected**: `exe`, `dll`, `sys`, `ocx`, `cpl`, `scr`, `drv`, `efi`, `com`, `msi` and the scripts `ps1`, `psm1`, `bat`, `cmd`, `vbs`, `vbe`, `js`, `jse`, `wsf`, `wsh`, `hta`;
- **collected as well**: Office documents **able to carry macros**, an intrusion vector as common as an executable — legacy binary formats (`doc`, `xls`, `ppt`, `pps`, `dot`, `xlt`, `pot`), "m" formats (`docm`, `dotm`, `xlsm`, `xltm`, `pptm`, `potm`, `ppsm`), `xlsb`, add-ins (`xla`, `xlam`, `xll`, `wll`, `ppa`, `ppam`), Publisher (`pub`), Visio (`vsd`, `vsdm`, `vstm`, `vssm`) and Access (`mdb`, `accdb`, `accde`). They appear in the traces as targets of shortcuts or jump lists, or among the files loaded by Word or Excel in their Prefetch;
- **only hashed**: other documents and data files, including `docx`, `xlsx` and `pptx`, which cannot contain macros. Copying them would turn the collection into a copy of the user's files.

The referenced files come from processes, services (binary and DLL), scheduled tasks, files loaded by programs (Prefetch), Shimcache, Amcache, and targets of shortcuts and jump lists. Each file is read only once, however many artefacts cite it, and **identical content is stored only once**: the other paths appear in the manifest as exhibits sharing that content (`Shared Exhibit`).

Authentic Microsoft binaries: hashed, not collected

Most referenced binaries are Windows components, identical on every machine of the same build. Their origin can be proven on the machine itself, with no list to carry: Windows' **catalogs**, signed by Microsoft, list the fingerprint of every system file. WAC reads those of `System32\CatRoot`, those of the components (`WinSxS\Catalogs`) and those of the servicing packages (`servicing\Packages`), over 13,000 on Windows 11. Individually signed binaries (Edge, OneDrive, Office, .NET) carry an **embedded signature**. A binary authenticated this way is hashed without being collected; every other one is collected — including a system binary replaced by an attacker, whose fingerprint no longer matches.

This check leaves **no trace**: catalogs and binaries are read directly from the disk, and the cryptographic verification is done in memory by WAC, up to Microsoft root certificates embedded in the executable. No service is solicited, the machine's certificate store is not consulted, and revocation is not checked online.

Only signers Microsoft uses for its own code are accepted ("Microsoft Windows", "Microsoft Corporation", "Microsoft Windows Publisher" and ".NET"). **Third-party** drivers certified by Microsoft (signers "Hardware Compatibility Publisher" and "Early Launch Anti-malware Publisher") are still collected: a vulnerable but signed driver is a classic attack path.

Store applications: verified through their package

The files of a Store application are not signed one by one: their **package** is. Its folder holds the signature (`AppxSignature.p7x`) and the block map (`AppxBlockMap.xml`), which gives the SHA-256 of every 64 KiB block of every file. WAC verifies the signature, which must reach a Microsoft root (the Store signs the packages it distributes), checks that it does cover that map, then compares every block read with the map. A matching file is hashed without being collected; a modified file, or one added to the folder after installation, is collected, and so is every file of a package signed outside the Microsoft chain.

The catalogs, package signatures and block maps that justified not collecting a file go into the exhibit store, so that a third party can redo the check. In the artefact files, the `Signature` field (or `ServiceDllSignature`, `SignatureTarget`…) states the authentication used, for instance "`Microsoft (catalog C:\Windows\….cat)`", "`Microsoft (embedded signature)`" or "`Package (package C:\Program Files\WindowsApps\…)`".

Scripts are authenticated the same way: by Windows' catalogs and, for PowerShell (`ps1`, `psm1`, `psd1`, `ps1xml`…), by their embedded signature block (`# SIG # Begin signature block`). A Windows Script Host script (`vbs`, `js`, `wsf`) signed only within its text, and not by a catalog, is still collected.

On the test machine, this check brings collection down from 2,131 binaries (3.3 GB) to 92 (335 MB), without lengthening the collection; the scripts still collected are the unsigned ones.

Since reading is direct, collecting leaves no more trace than hashing — it only takes space on the medium.

4.6 What the screen shows

WAC displays each step, followed by `OK` or an error message. A failed step does not stop the collection: whatever is accessible is gathered, and whatever is missing is recorded.

```
[PREREQUISITE VERIFICATION]
 - Check administrator rights : OK
 - Checking the collection medium : OK
[OBSERVING THE RUNNING MACHINE]
 - Extraction of SESSIONS : OK
 - Extraction of PROCESS : OK
 - Extraction of NETWORK STATE : OK
[RAW EXTRACTION]
 - Extracting system hives (raw NTFS) : OK
 - Opening the SYSTEM hive : OK
 - Opening the SOFTWARE hive : OK
 - Listing USER PROFILES (SOFTWARE hive) : OK
 - Extracting user hives (raw NTFS) : OK
 - Extracting file artefacts (raw NTFS) : OK
[REGISTRY ARTEFACTS]
 - Extracting USERASSIST Registry Keys : OK
   ...
[FILE ARTEFACTS]
 - Extracting PREFETCHS : OK
```

```
   ...
[EVENT LOGS]
 - Extraction of EVENTS : OK
[EXHIBIT STORE]
 - Copying collected binaries to the working directory : OK
 - Sealing the exhibit store (manifest + SHA-256) : OK
[INVESTIGATION LOG]
 - Writing investigation.json : OK
END, Time elapsed : 309 s
```

This is the display with `--language=en`, or `language: en` in `wac.yml`: by default the console is in French.

Sealing the exhibit store is the **last** operation on it: every exhibit, including those added while the event logs are processed, is covered by the manifest.

4.7 Duration

The duration depends mostly on the hardware (USB 2 or 3, disk, processor) and the options. On the test virtual machine (NVMe disk): about 70 s with `--events`, about 310 s with `--events` and `--binary`. On a USB stick, writing dominates.

CHAPTER 5

Collect here, convert elsewhere

5.1 Why separate

A run of WAC does two different things:

1. **the collection**: observing what only a running system shows (processes, sessions, service states, clock), reading the disk directly, copying the evidence into a sealed exhibit store. It **needs** the examined machine;
2. **the conversion**: parsing thousands of copied files (hives, event logs, Prefetch, shortcuts…) into JSON. It only needs the exhibit store, **not** the machine.

WAC **never does both in the same run**: it collects on the examined machine, and converts elsewhere. Converting on the machine would add work there that nothing requires: minutes of processing in WAC's memory, which Windows may page out to `pagefile.sys`, on the very disk under examination; a process present for longer, seen and logged by the machine's security software; if it crashes, a Windows error report written on the machine; and system libraries loaded to decode (the ESE engine, to replay a database's logs).

That is exactly what good practice asks to avoid:

- **ISO/IEC 27037** covers the identification, collection and preservation of digital evidence, **ISO/IEC 27042** its analysis: two distinct stages;
- **ACPO principle 1**: no action should change the data of the examined machine; every operation run there must be necessary;
- **ACPO principle 3**: an independent third party must be able to repeat the process and reach the same result.

Separating the two stages brings three things:

- **fewer traces** on the examined machine: only the collection runs there;
- **evidence checked before any analysis**: `--convert` first verifies the seal and the SHA-256 of every exhibit, and refuses a retouched collection;
- **a reproducible analysis**: two conversions of one collection give byte-identical JSON files, and the analysis can be redone later, with a corrected version of WAC or by a third party, without going back to the machine.

5.2 The collection's settings

- **Scope of `--binary`**: the collection reads **every executable of every fixed NTFS volume**, cited or not, since it does not know yet which ones the artefacts will cite. That time (8 min 30 on the test machine) is reading, which writes nothing to the machine.
- **`--binary-all`**: when the content of every executable must be in the exhibit store (malware analysis, request for the files themselves), for about 20 GB.

5.3 The two commands

On the examined machine, as administrator:

```
E:\> WAC.exe --collect --events --binary --output=accounting-pc-01
```

On the analysis workstation, with no particular rights, with the same configuration (`events`, `--binary`) as the collection:

```
D:\> WAC.exe --convert=E:\accounting-pc-01 --events --binary
```

The JSON files are written into the collection's folder, next to the exhibit store. `--convert` excludes `--collect` and `--output`.

5.4 What `--collect` takes

Since the conversion can no longer ask the machine anything, the collection takes **everything it will read**:

- the **live observations**, recorded as read in `exhibits\live` and sealed like any exhibit: processes, sessions, service states, clock and boot, and the machine's settings (system drive, code page, time zone, volumes and their serial numbers);
- the hives and the file artefacts, as usual;
- with the event logs (`events`), the resource files of **every** declared event provider (binary, parameter file, `.mui` satellites): which of them are needed is only known at conversion. About 650 MB on Windows 11;
- with `--binary`, **every** executable, library, driver, script and macro document of **every fixed NTFS volume** — a profile on `D:` included —, read directly and verified in memory (section 4.5):
  - authenticated (catalog, embedded signature, Store package), or a third-party binary whose signature holds against the trust set of the key (`--update-trust`: chain to a root Microsoft trusts for code signing, valid at the signing time, not revoked, not a vulnerable driver): **not copied**; the manifest keeps its path, its Authenticode digests and the verdict (`ContentStored false`); without a usable trust set, every third-party binary is copied, as the investigation log says;

  - not authenticated: **copied**, with its three fingerprints MD5, SHA-1, SHA-256, its Authenticode digest and the reason;
  - Windows differential file (`PA30`/`PA31`, the `r` and `f` folders of `WinSxS`): not an executable; hashed, not copied.

A content is stored once, and the other names of a hard link (`System32` and `WinSxS`) are recorded in the manifest like the first. Removable volumes, the collection medium among them, and the memory files (`pagefile.sys`, `swapfile.sys`, `hiberfil.sys`) are left out.

> **Warning**
>
> With `--binary`, the collection reads every volume: on the test virtual machine (Windows 11 without added software), 44,446 executables read, of which 41,593 authenticated (or third-party with a valid signature) not copied and 2,853 copied (1.2 GB); the whole collection takes 8 minutes 30 and the exhibit store, event logs and NTFS metadata included, weighs 2.7 GB (see section 3.2). If space runs short, the remaining files are hashed without being copied, and the manifest says so.

No artefact JSON file is written: the folder holds only the sealed exhibit store, the working directory and `investigation.json`.

5.5 What `--convert` does

1. **Check**, before any read: the seal must carry the manifest's real fingerprint, and every exhibit its SHA-256 fingerprint. A single altered exhibit, and the conversion is **refused**; `conversion.json` gives the reason.
2. **Rebuild of the working directory** from the exhibit store: the former one is deleted rather than trusted, the copies are verified against the manifest and the hives repaired, as at collection time. The executables and the providers' resources are read in the exhibit store itself, without a copy.
3. **Conversion**, from the exhibits alone: nothing is read from the converting machine — neither its disk, nor its registry, nor its time zone.

The exhibit store is never modified and its seal remains the collection's. `investigation.json` remains the collection's log; `conversion.json` is the conversion's (operator, workstation, operations). Two conversions of one collection give identical JSON files, which the test harness checks on every cycle.

5.6 Limitations

- **Macro documents** (`doc`, `docm`, `xls`, `xlsm`, `ppt`…, list in section 4.5) are handled like executables: read on every fixed volume and **collected**, unless they are, byte for byte, a file shipped by Microsoft and listed in a catalog — which a document written or modified by anyone cannot be.

- A cited file that can carry neither code nor macro (`docx`, `xlsx`, `pdf`, data) is not collected: at conversion it therefore has **no fingerprint**.
- An authenticated executable is not copied: at conversion it has its Authenticode digest and its verdict, but no MD5, SHA-1 or SHA-256 — those are computed only for what is collected.
- An executable on a removable volume is not collected by `--collect --binary`.
- The configuration (`events`, `--binary`) of the conversion must be that of the collection: what the collection did not take cannot be converted.

CHAPTER 6

The results

6.1 Layout

```
<output>/
  exhibits/            exhibit store: raw copies, never rewritten
    MANIFEST.json     identifies each exhibit and the collection
    MANIFEST.sha256   the manifest's fingerprint: its seal
    Windows/...        the exhibits, under their original paths
    Users/...
  working/             working copies (replayed hives)
  investigation.json   the collection log
  *.json               the artefacts
```

A file from a volume other than the system one is stored under `_volume_D\...`; the JSON always gives the original path, with its real drive letter.

Under `exhibits/` and `working/`, a path often passes 260 characters: the original one, plus the output folder. WAC writes it in the extended form (`\\?\`). Explorer and several Windows commands (`Copy-Item`, `Remove-Item` of PowerShell 5.1, a drag-and-drop copy) stop on these files, sometimes silently. To copy or delete a collection, use `robocopy` or `rd /s /q \\?\<path>`; to check it, `verify-exhibits.ps1`, which reads a long-path exhibit in the extended form.

6.2 The exhibit store manifest

`exhibits/MANIFEST.json` identifies the collection and each exhibit.

6.2.1 Header

`Tool`
: tool name, command line, build date.

`Host`
: machine name, system drive, time zone of the suspect (read from its `SYSTEM` hive) and of the collecting machine.

`Operator`
: account that ran the collection, and whether it was elevated.

`Custody`
: procedure statement, start of extraction, time the manifest was written, number of

exhibits and failures, total size, hash algorithms, volumes read (letter, serial number, file system).

6.2.2 Each exhibit

Field	Meaning
`SourcePath`	Original path, with drive letter.
`ExhibitPath`	Location of the raw copy, relative to the output folder.
`Method`	Collection method (direct NTFS reading, volume, purpose).
`Result`	`OK`, or the error code and message. Failures are listed in the manifest: a missing exhibit would read as one never looked for.
`MD5, SHA1, SHA256`	The three fingerprints, computed while writing: they bear on what was read from the volume.
`Bytes`	Extracted size.
`DeclaredBytes`	Size declared by NTFS, present only when it differs from the extracted size (truncated extraction).
`ValidDataBytes`	Valid data length of a preallocated file (event logs): beyond it, the exhibit holds zeros by definition.
`SharedExhibit`	`true` when the content, identical byte for byte, is already stored under another exhibit: `ExhibitPath` points to it. The source remains an exhibit in its own right, with its own path, `$MFT` entry and timestamps.
`MftEntry`	NTFS record number: identifies the file independently of its name.
`Extracted(Utc)`	When this exhibit was extracted.
`SourceCreated, SourceModified, SourceMftModified, SourceAccessed`	The four NTFS timestamps of the original file (`$STANDARD_INFORMATION`), in UTC and in the suspect's local time. They, not the dates of the copies, date the file.
`SourceFileName Created, Modified, MftModified, Accessed`	The four timestamps of the `$FILE_NAME` attribute of the name read. NTFS keeps them apart, and a change of the dates by a program leaves them untouched: a `Source Created` earlier than `SourceFileNameCreated` is the classic sign of dates forged after the fact (*timestomping*).
`ContentStored`	`false`: authenticated binary read but not copied (`--collect --binary`); it has no `ExhibitPath`.
`AuthenticodeSHA1, AuthenticodeSHA256`	Authenticode digests of a PE, those its signature and the catalogs cover; the only digests of an authenticated binary that was not copied.

Field	Meaning
`SignatureVerified`	Outcome of an executable's signature check: `true` with `Signature` (what authenticates it), `false` with `Signature Reason` (why not).
`PeTimeDateStamp`, `PeSizeOfImage`, `SymbolServerKey`	Build of a PE and the key (`name/TIMESTAMPsize/name`) under which Microsoft's symbol server keeps it: an authentic Microsoft binary that was not copied can be fetched again from it, identical, and its fingerprint checked.

> **Why three fingerprints**
> MD5 collisions have been producible at will since 2008, and SHA-1 ones since 2017: a single fingerprint no longer settles an identity dispute. Three independent ones do.

6.3 Verifying a collection

6.3.1 The seal

The manifest cannot hold its own fingerprint: it is in `MANIFEST.sha256`. On Linux:

```
$ cd output/exhibits
$ sha256sum -c MANIFEST.sha256
MANIFEST.json: OK
```

The seal written down. `MANIFEST.sha256` proves only that the manifest is unchanged since it was sealed: whoever rewrites the manifest can rewrite its seal. `WAC` therefore prints the seal on the screen at the end of the collection ("Seal of the manifest (SHA-256), to write down in the report"): written down in the report, it is the one copy out of reach of the collection, and the scripts below hold the manifest against it (`--seal` / `-Seal`).

6.3.2 Each exhibit

Two scripts, delivered with `WAC.exe` in `bin`, check the seal, then the SHA-256 of every exhibit: `verify-exhibits.ps1` on Windows, `verify-exhibits.sh` on Linux. `verifier-consigne.ps1` and `verifier-consigne.sh` run the same scripts with French messages. A manifest without its list of exhibits is refused, never taken for an empty collection; an exhibit that cannot be read is counted apart from one that differs. The PowerShell scripts need PowerShell 4 or later (Windows 8.1 onwards): on an older PowerShell they say so and stop (code 2) rather than read wrongly.

Verify from a Linux machine. This is the recommended way: the collection may hold what infected the examined machine, and a Linux investigation machine is a technological break — a Windows executable or macro document of the collection cannot run there. The

script needs only `bash`, the coreutils and `python3` (which reads the JSON); the fingerprints are computed by `sha256sum`, not by WAC:

```
$ bash verify-exhibits.sh --seal=CEB489A1...8A5B67 /media/usb/output
Seal verified (CEB489A143AE29EB9E3BC836F3D565B744F036AE1F3A72A3447408F31F8A5B67)
1617 exhibit(s) verified, 0 mismatch(es)
0 binary(ies) authenticated and fingerprinted only (no copy to verify)
```

The manifest comes from the examined machine: both scripts refuse an exhibit path that leaves the collection (`..`, absolute path, and for the `.sh` a symbolic link), and count it as a mismatch without reading it.

From Windows:

```
PS> powershell -ExecutionPolicy Bypass -File verify-exhibits.ps1 -Output E:\output -Seal
    CEB489A1...8A5B67
Seal verified (CEB489A143AE29EB9E3BC836F3D565B744F036AE1F3A72A3447408F31F8A5B67)
1617 exhibit(s) verified, 0 mismatch(es)
```

Their exit code is 0 if everything matches, 1 otherwise.

```
#!/usr/bin/env bash
# verify-exhibits.sh — checks the seal and every exhibit of a WAC collection,
# from a Linux investigation machine: the collection is examined away from the
# Windows machine it comes from, which it cannot infect there.
# Usage: bash verify-exhibits.sh [--language=en|fr] [--seal=<SHA-256>] /media/usb/output
#   --seal      the seal WAC printed at the collection, as written down in the
#               report: compared with MANIFEST.sha256 (see below)
#   --language  the messages' language; verifier-consigne.sh is this script
#               in French
# Exit code: 0 if everything matches, 1 otherwise.
# Needs: bash, coreutils (sha256sum, realpath), python3 (reads the JSON only).
#
# The fingerprints are computed by sha256sum, not by WAC: a verification that
# does not rely on the tool that produced the collection.
#
# The seal beside the manifest proves only that the manifest is unchanged
# since it was sealed: whoever rewrites the manifest can rewrite its seal.
# Against that, the value written down at the collection (--seal).
#
# The manifest comes from the examined machine, so it is treated as hostile:
# an exhibit path that leaves the collection (.., absolute path, symbolic
# link) is refused and counted as a mismatch, never read; a manifest without
# its list of exhibits is refused, never taken for an empty collection.

set -u
language=en; expected_seal=""; output="./output"
for argument in "$@"; do
    case "$argument" in
        --language=*) language="${argument#--language=}" ;;
        --seal=*) expected_seal="$(printf '%s' "${argument#--seal=}" | tr 'a-f' 'A-F')" ;;
        *) output="$argument" ;;
    esac
done
```

```
# The messages, in the language asked for. $1: the message's key; the rest
# goes into its text.
say() {
    local key="$1"; shift
    if [ "$language" = fr ]; then
        case "$key" in
            tool) echo "OUTIL ABSENT : $1" ;;
            absent) echo "MANIFESTE ABSENT : $1 ou $2" ;;
            seal) echo "SCEAU NON CONFORME : manifeste modifie" ;;
            noted) echo "SCEAU NON CONFORME : $1 au lieu de $2, releve a la collecte" ;;
            sealed) echo "Sceau conforme ($1)" ;;
            unnoted) echo "Sceau compare a MANIFEST.sha256 seul : donner --seal=<valeur
    relevee a la collecte> pour l'opposer a une reecriture" ;;
            unreadable) echo "MANIFESTE ILLISIBLE : $1" ;;
            mismatch) echo "DIFFERENTE : $1" ;;
            total) echo "$1 piece(s) conforme(s), $2 differente(s)" ;;
            binaries) echo "$1 binaire(s) authentifie(s), empreinte seule (pas de copie a
    verifier)" ;;
        esac
    else
        case "$key" in
            tool) echo "MISSING TOOL: $1" ;;
            absent) echo "NO MANIFEST: $1 or $2 absent" ;;
            seal) echo "SEAL MISMATCH: the manifest was modified" ;;
            noted) echo "SEAL MISMATCH: $1 instead of $2, written down at the collection"
    ;;
            sealed) echo "Seal verified ($1)" ;;
            unnoted) echo "Seal checked against MANIFEST.sha256 only: give --seal=<value
    written down at the collection> to hold it against a rewrite" ;;
            unreadable) echo "UNREADABLE MANIFEST: $1" ;;
            mismatch) echo "MISMATCH: $1" ;;
            total) echo "$1 exhibit(s) verified, $2 mismatch(es)" ;;
            binaries) echo "$1 binary(ies) authenticated and fingerprinted only (no copy
    to verify)" ;;
        esac
    fi
}

manifest="$output/exhibits/MANIFEST.json"
seal="$output/exhibits/MANIFEST.sha256"
for tool in sha256sum realpath python3; do
    command -v "$tool" >/dev/null || { say tool "$tool"; exit 1; }
done
[ -f "$manifest" ] && [ -f "$seal" ] || { say absent "$manifest" "$seal"; exit 1; }
root="$(realpath -e -- "$output")" || exit 1

# 1. The seal: the manifest's fingerprint must be the one recorded beside it,
# and the one written down at the collection when it is given.
recorded="$(head -c 64 -- "$seal" | tr 'a-f' 'A-F')"
actual="$(sha256sum -- "$manifest" | head -c 64 | tr 'a-f' 'A-F')"
if [ "$recorded" != "$actual" ]; then say seal; exit 1; fi
if [ -n "$expected_seal" ] && [ "$expected_seal" != "$actual" ]; then say noted "$actual"
```

```
    "$expected_seal"; exit 1; fi
say sealed "$actual"
[ -n "$expected_seal" ] || say unnoted

# 2. Every exhibit: its SHA-256 must be the one in the manifest.
# python3 lists the exhibits kept with their content, as NUL-separated
# "S, SHA-256, path" records, and one "F" record per authenticated binary that
# was not copied (--collect --binary): no file to verify. NUL cannot occur in
# a path, whatever the manifest holds. A manifest that is no JSON, or whose
# Items is no list, ends with exit code 3: refused.
list_exhibits() {
    python3 - "$manifest" <<'PYTHON'
import json, sys
try:
    with open(sys.argv[1], encoding="utf-8-sig") as f:
        items = json.load(f).get("Items")
except (OSError, ValueError, AttributeError):
    sys.exit(3)
if not isinstance(items, list):
    sys.exit(3)
out = sys.stdout.buffer
for item in items:
    if not isinstance(item, dict) or item.get("Result") != "OK":
        continue
    if item.get("ContentStored") is False:
        out.write(b"F\0")
        continue
    sha = str(item.get("SHA256") or "").replace("\0", "")
    path = str(item.get("ExhibitPath") or "").replace("\0", "")
    out.write(b"S\0" + sha.encode() + b"\0" + path.encode("utf-8", "surrogateescape") + b
    "\0")
PYTHON
}

ok=0; ko=0; fingerprint_only=0
while IFS= read -r -d '' kind; do
    if [ "$kind" = F ]; then fingerprint_only=$((fingerprint_only + 1)); continue; fi
    IFS= read -r -d '' expected || break
    IFS= read -r -d '' exhibit || break
    if [ "$kind" = X ]; then ko=$((ko + 1)); say unreadable "$manifest"; continue; fi
    # The manifest writes Windows paths, relative to the output folder.
    relative="${exhibit//\\//}"
    file="$(realpath -e -- "$root/$relative" 2>/dev/null)" || file=""
    case "$relative" in
        ""|/*) file="" ;;
    esac
    case "$file" in
        "$root"/*) ;;
        *) ko=$((ko + 1)); say mismatch "$exhibit"; continue ;;
    esac
    hash="$(sha256sum -- "$file" | head -c 64 | tr 'a-f' 'A-F')"
    if [ -n "$expected" ] && [ "$hash" = "$(printf '%s' "$expected" | tr 'a-f' 'A-F')" ];
    then
```

```
        ok=$((ok + 1))
    else
        ko=$((ko + 1)); say mismatch "$exhibit"
    fi
done < <(list_exhibits || printf 'X\0\0\0')

say total "$ok" "$ko"
say binaries "$fingerprint_only"
[ "$ko" -eq 0 ] || exit 1
```

```
# verify-exhibits.ps1 — checks the seal and every exhibit of a WAC collection.
# Usage: powershell -ExecutionPolicy Bypass -File verify-exhibits.ps1 -Output E:\output [-
    Seal <SHA-256>] [-Language fr]
#   -Seal      the seal WAC printed at the collection, as written down in the
#              report: compared with MANIFEST.sha256 (see below)
#   -Language  the messages' language; verifier-consigne.ps1 is this script in French
# Exit code: 0 if everything matches, 1 otherwise, 2 if PowerShell is older than 4.
#
# The seal beside the manifest proves only that the manifest is unchanged
# since it was sealed: whoever rewrites the manifest can rewrite its seal.
# Against that, the value written down at the collection (-Seal).

param([string]$Output = '.\output', [string]$Seal = '', [string]$Language = 'en')

# The messages, in the language asked for: {0}, {1} take the values.
$messages = @{
    en = @{
        Version    = 'PowerShell 4 or later is needed (this is {0}): use verify-exhibits.
    sh, or a more recent Windows'
        Absent     = 'NO MANIFEST: {0} or {1} absent'
        Seal       = 'SEAL MISMATCH: the manifest was modified'
        Noted      = 'SEAL MISMATCH: {0} instead of {1}, written down at the collection'
        Sealed     = 'Seal verified ({0})'
        Unnoted    = 'Seal checked against MANIFEST.sha256 only: give -Seal <value written
     down at the collection> to hold it against a rewrite'
        Unreadable = 'UNREADABLE MANIFEST: {0}'
        Mismatch   = 'MISMATCH: {0}'
        NotRead    = 'UNREADABLE EXHIBIT: {0}'
        Total      = '{0} exhibit(s) verified, {1} mismatch(es)'
        Binaries   = '{0} binary(ies) authenticated and fingerprinted only (no copy to
    verify)'
    }
    fr = @{
        Version    = 'PowerShell 4 ou plus est nécessaire (version {0}) : utiliser
    verifier-consigne.sh, ou un Windows plus récent'
        Absent     = 'MANIFESTE ABSENT : {0} ou {1}'
        Seal       = 'SCEAU NON CONFORME : manifeste modifié'
        Noted      = 'SCEAU NON CONFORME : {0} au lieu de {1}, relevé à la collecte'
        Sealed     = 'Sceau conforme ({0})'
        Unnoted    = 'Sceau comparé à MANIFEST.sha256 seul : donner -Seal <valeur relevée
    à la collecte> pour l''opposer à une réécriture'
        Unreadable = 'MANIFESTE ILLISIBLE : {0}'
        Mismatch   = 'DIFFÉRENTE : {0}'
```

```
        NotRead    = 'PIÈCE ILLISIBLE : {0}'
        Total      = '{0} pièce(s) conforme(s), {1} différente(s)'
        Binaries   = '{0} binaire(s) authentifié(s), empreinte seule (pas de copie à vé
    rifier)'
    }
}
$text = if ($messages.ContainsKey($Language)) { $messages[$Language] } else { $messages['
    en'] }
function Say([string]$key) { Write-Output ($text[$key] -f $args) }

# Get-FileHash, which reads the seal, came with PowerShell 4:
# on an older one the reading would fail, and its failure read as a mismatch
# of the seal. Said so instead.
if ($PSVersionTable.PSVersion.Major -lt 4) { Say Version $PSVersionTable.PSVersion; exit 2
     }

# Both files first: a missing one is no tampering, and must not read as one.
$manifest = "$Output\exhibits\MANIFEST.json"
$sealFile = "$Output\exhibits\MANIFEST.sha256"
if (-not (Test-Path -LiteralPath $manifest -PathType Leaf) -or -not (Test-Path -
    LiteralPath $sealFile -PathType Leaf)) {
    Say Absent $manifest $sealFile; exit 1
}

# 1. The seal: the manifest's fingerprint must be the one recorded beside it,
# and the one written down at the collection when it is given.
$recorded = ((Get-Content -Raw -LiteralPath $sealFile) -split '\s+')[0]
$actual   = (Get-FileHash -Algorithm SHA256 -LiteralPath $manifest).Hash
if ($recorded -ne $actual) { Say Seal; exit 1 }
if ($Seal -and $Seal -ne $actual) { Say Noted $actual $Seal; exit 1 }
Say Sealed $actual
if (-not $Seal) { Say Unnoted }

# 2. Every exhibit: its SHA-256 must be the one in the manifest. A manifest
# that is no JSON, or without its list of exhibits, is refused: never taken
# for an empty collection. Read by the serializer ConvertFrom-Json is built
# on, its limit lifted as PowerShell 5 does: that of PowerShell 4, 2 MB of
# text, refused every collection's manifest (tens of MB) on Windows 8.1.
try {
    Add-Type -AssemblyName System.Web.Extensions
    $serializer = New-Object Web.Script.Serialization.JavaScriptSerializer
    $serializer.MaxJsonLength = [int]::MaxValue
    $m = $serializer.DeserializeObject((Get-Content -Raw -Encoding UTF8 -LiteralPath
    $manifest))
} catch { $m = $null }
if ($m -isnot [Collections.IDictionary] -or $m['Items'] -isnot [Collections.IList]) { Say
    Unreadable $manifest; Say Total 0 1; exit 1 }
# The manifest comes from the examined machine: an exhibit path that leaves
# the collection (.., absolute path, drive) is refused, never read. Checked on
# the string.
#
# Each exhibit is opened by CreateFileW and read through its handle: a path
# past 259 characters needs \\?\, which .NET refuses before 4.6.2 (Windows
```

```
# 8.1 has 4.5) and CreateFileW takes on every Windows. Below that, an
# ordinary path, normalised as WAC's own writing was.
Add-Type -TypeDefinition @'
using System; using System.Runtime.InteropServices; using Microsoft.Win32.SafeHandles;
public static class ExhibitFile {
    [DllImport("kernel32.dll", CharSet = CharSet.Unicode, SetLastError = true)]
    public static extern SafeFileHandle CreateFileW(string name, uint access, uint share,
    IntPtr security,
                                                    uint disposition, uint flags, IntPtr
    template);
}
'@
$GENERIC_READ = [uint32]2147483648   # 0x80000000: as a literal, a negative Int32 before
    PowerShell 7
$FILE_SHARE_READ = 1; $OPEN_EXISTING = 3; $FILE_FLAG_SEQUENTIAL_SCAN = 0x08000000
$sha256 = [Security.Cryptography.SHA256]::Create()
# The SHA-256 of a file, in upper-case hexadecimal; throws if it cannot be read.
function Get-ExhibitHash([string]$path) {
    $handle = [ExhibitFile]::CreateFileW($path, $GENERIC_READ, $FILE_SHARE_READ, [IntPtr
    ]::Zero,
                                         $OPEN_EXISTING, $FILE_FLAG_SEQUENTIAL_SCAN, [
    IntPtr]::Zero)
    if ($handle.IsInvalid) { throw "CreateFileW: $([Runtime.InteropServices.Marshal]::
    GetLastWin32Error())" }
    $stream = New-Object IO.FileStream($handle, [IO.FileAccess]::Read)
    try { [BitConverter]::ToString($sha256.ComputeHash($stream)) -replace '-', '' }
    finally { $stream.Dispose() }
}
$root = (Resolve-Path -LiteralPath $Output).ProviderPath.TrimEnd('\') + '\'
$ok = 0; $ko = 0; $fingerprintOnly = 0
foreach ($p in ($m.Items | Where-Object { $_.Result -eq 'OK' })) {
    # An authenticated binary that was not copied (--collect --binary): no file.
    if ($p.ContentStored -eq $false) { $fingerprintOnly++; continue }
    $rel = ([string]$p.ExhibitPath) -replace '/', '\'
    if (-not $rel -or [IO.Path]::IsPathRooted($rel) -or $rel -match ':' -or ($rel -split
    '\\') -contains '..') {
        $ko++; Say Mismatch $p.ExhibitPath; continue
    }
    $path = $root + $rel
    if ($path.Length -gt 259) { $path = if ($root.StartsWith('\\')) { '\\?\UNC\' + $path.
    Substring(2) } else { '\\?\' + $path } }
    try { $h = Get-ExhibitHash $path }
    catch { $ko++; Say NotRead $p.ExhibitPath; continue }   # unread is no mismatch: said
    as such
    if ($h -eq $p.SHA256) { $ok++ } else { $ko++; Say Mismatch $p.ExhibitPath }
}
Say Total $ok $ko
Say Binaries $fingerprintOnly
if ($ko) { exit 1 }
```

6.4 The investigation log

`investigation.json` traces the collection: tool and command line, machine and time zones, operator, start, end and duration, then the **ordered** list of operations. For each one:

`Seq`, `TimestampUtc`, `TimestampLocal`
: rank and time;

`Operation`, `Target`
: what was done, and on what;

`Result`
: the outcome;

`Footprint`
: the trace this operation is expected to leave on the examined machine.

This log makes it possible to compare, collection by collection, the expected traces with those actually observed. It also flags a difference between the suspect's time zone and the collecting machine's.

6.5 The artefact files

One file per artefact. The fields described are those emitted; a field without a value is not emitted (see chapter 7).

6.5.1 System and accounts

`OperatingSystem.json`
: machine name and domain, Windows edition and version, installation date, current time, **boot time**, uptime, time zone. `BootTimeSource` says where the boot time comes from (kernel, or fallback estimate); `ClockAdjustedSinceBootMs` gives the total clock corrections since boot — a few seconds is routine resynchronisation, a large value means a clock changed. `ShutdownTime`: the last orderly shutdown. `NtfsLastAccessUpdated`: false, NTFS does not update last access dates, which then say nothing. `Settings`: the settings that decide how the other artefacts read, as stored — NTFS last access and short names, Prefetch on, page file and memory dumps, control sets, remote desktop, Windows timeline, last user who logged on.

`security.json`
: `SECURITY` hive: **names** of the LSA secrets and the dates of their values (`DefaultPassword` reveals an automatic logon's stored password, `_SC_<service>` a service running under an account), number of cached domain logon slots and how many are used, audit policy (`PolAdtEv`, as bytes: its layout is not published), and the account rights (`AccountRights`, read from `Policy\Accounts`): for each account or group, its privileges (`Privileges`, e.g. `SeDebugPrivilege`, `SeBackupPrivilege`) and its logon rights (`LogonRights`). A sensitive privilege

granted to an unexpected account is a persistence. **No secret's value** is read, nor any cached credential. The `config\RegBack` copies of the hives are collected as they are.

`users.json`
local accounts, read from the `SAM` hive: SID and RID, state, flags, logon and failure counts, dates of last logon, last failure, password change and account change, profile, and the password's NT and LM hashes (`NtHash`, `LmHash`, hex), with their history (`NtHashHistory`, `LmHashHistory`). A hash is one-way: it names a password without revealing it. A weak or empty password used by an intruder reads as a legitimate logon; **WAC judges nothing**, it converts, and WAE confronts the hashes, offline, with those of known weak passwords. The hashes are decrypted on the spot (the boot key from `SYSTEM`, the hashed boot key from the `SAM`, the AES scheme of Windows 10 and later or the older RC4, whose checksum is verified: a wrong key gives no hash rather than wrong ones, then a DES layer keyed on the RID, which WAC decrypts itself so that it holds from XP to Windows 11). If the account carries password-reset security questions (`SecurityQuestions`, the `ResetData` value), they are converted as they are, questions AND answers: stored in the clear, a trivial answer is a weakness WAE judges like a weak password.

`local_groups.json`
local groups read from the `SAM`: the built-in groups (`Scope Builtin`, `S-1-5-32-…`) and the groups created on the machine (`Local`), each with its RID, SID, name, description and direct members (SID and name resolved offline). Membership is what says who is an administrator or a remote desktop user; an account an intruder adds to a privileged group shows up here.

`Sessions.json`
sessions open at collection time: user, logon type, authentication package, start time.

`processes.json`
running processes: name, identifiers (and parent), owner, session, loaded modules and, with `--binary`, the executable's fingerprints.

`network.json`
network state at collection time: `Connections` (TCP and UDP, IPv4 and IPv6: addresses, ports, state, owning process and its name, creation time local and UTC), `Neighbours` (ARP and NDP caches: the machines of the local network spoken with lately), `Interfaces` (addresses, gateways, DNS servers, DHCP) and `Routes`. `TablesNotRead` names a table Windows refused to give, which is not an empty table. `TablesIpv4Only` names the tables read through the functions of Windows XP and Server 2003, which know IPv4 only: no IPv6 row there does not mean no IPv6 connection. On these versions a connection has no creation time, and a neighbour carries a `Type` (dynamic or static ARP entry) in place of a `State`.

`services.json`
services and drivers, configuration read from the hive: type, start mode, account, binary and hosted DLL, failure command, dependencies, last write of the key; current state; with `--binary`, fingerprints of the binary and of the DLL.

`ScheduledTasks.json`
scheduled tasks, read from their XML definitions: author, run-as account, state, last run and its result, triggers and actions (with `--binary`, fingerprints of the command). On

Windows XP and Server 2003, from their `.job` files (`SourceJob`): program, parameters, working directory, author, comment, last run (local time converted), exit code, triggers, and the scheduler's status (`JobStatus`: `HasNotRun`, `Ready`…); the account a task runs as is not in the file.

6.5.2 Program execution

`prefetchs.json`
Prefetch files: program, run count and times, volumes, and list of loaded files with their `$MFT` reference (and their fingerprints with `--binary`). A missing reference is not a failed read: Windows writes it as zero, for every file of a program run only once and for some of the others. Versions decoded: 17 (Windows XP, which records no `$MFT` reference and one run time), 23 (Vista and 7, one run time), 26 (Windows 8.1), 30 and 31 (Windows 10 and 11, compressed); the others are declared "unsupported format" (section 7.10).

`amcache_application_files.json`, `amcache_applications.json`, `amcache_inventory.json`
Amcache inventory, table by table and value by value, with the last write time of each entry: the executables (path, publisher, version, size, link date, `AmcacheSha1`), the installed applications, and in the third file the drivers (with their `AmcacheSha1`), devices, containers, driver packages and shortcuts, each with its `Table`. **Beware**: for a file larger than 30 MiB, Amcache hashes only the first 31,457,280 bytes; `AmcacheSha1CoversBytes` says so, and that SHA-1 does not compare with the whole file's. On Windows 8, 8.1 and the first Windows 10, the Amcache is made of other tables, `File` and `Programs`, whose values are numbered, not named: WAC names only those whose meaning was confirmed — for a file, `LongPath`, `AmcacheSha1`, `ProgramId`, its `$MFT` reference (`MftReference`, read in its key's name) and its volume's GUID; for a program, `Name`, `Version`, `Publisher`, `UninstallKeys` and `Files` —, and gives the others as read (`UnnamedValues`: name, type, value). On Windows 7, the executables run since the last pass of its inventory task, listed in `RecentFileCache.bcf`, are given in `amcache_application_files.json` with `Table` `RecentFileCache`: their path only, the file holds no date.

`shimcache.json`
compatibility cache: path and modification date of the file, position in the cache (`CachePosition`, 0 for the most recent). An entry proves the binary was present, not that it ran. The execution flag other tools publish is *not* collected: measured against Prefetch, its "yes" is nearly always right, but its "no" is wrong for 9 executed binaries out of 10 — it would lead to conclude that a program never ran. Formats decoded: Windows XP, Vista, 7, 8, 8.1, 10 and 11; on Windows 8.1 the path starts with `SYSVOL\`, the system volume, and is published as it is. Windows XP also records the file's size (`FileSize`) and when it updated the entry (`LastUpdateUtc`). Server 2003 writes Vista's signature (`0xBADC0FFE`) with other contents: the examined version tells them apart, and Server 2003's form, not measured yet, is declared unsupported.

`bams.json`
last execution per program and per user (Background Activity Moderator). The hives being read from the disk, an execution of the last minute or so may be missing: Windows keeps a registry change in memory before writing it (56 to 82 s measured on the test machine, which `RegFlushKey` did not bring forward), and WAC does not force that write, which would

modify the examined disk.

`userassists.json`
programs launched from the user interface: count, number and duration in the foreground (`FocusTimeMs`), last execution. On Windows XP the value keeps no foreground, and its counter starts at 5 (measured: two launches read 7): `Count` is the counter less 5.

`muicache.json`
display names of executed programs (on Windows XP, in `ShellNoRoam\MUICache` of NTUSER.DAT).

`run.json`
programs started at boot or logon (Run and RunOnce keys), with the key's last write: in SOFTWARE (the whole machine), in the DEFAULT hive (the system account) and in each profile's NTUSER.DAT (`Hive`). `Type` `REG_EXPAND_SZ`: the command holds variables (`%SystemRoot%`…), given as written. Also: the 32-bit programs' Run keys (`SOFTWARE (32-bit)`); `StartupApproved` (`Key` `StartupApproved\Run`, `Run32`, `StartupFolder`), an entry's state as the user set it in Task Manager or Settings: `Status`, the first 4 bytes as stored (their meaning is not measured, so not interpreted), and `Changed`, the instant of the change; `RunNotification`, a startup entry Windows noticed (`Notification`). A name without a matching Run value is an entry since removed.

`persistence.json`
registry persistence beyond the Run keys: IFEO (`Debugger`, `GlobalFlag`), SilentProcessExit, AppInit_DLLs and AppCertDlls, Winlogon (`Shell`, `Userinit`, `Taskman`, `Notify`), LSA packages, print monitors, `BootExecute`, Active Setup, NetSh helpers, the open commands of the executable file types (`exefile`, `batfile`, `cmdfile`…, machine and user), and COM hijacking in each user's classes (UsrClass.dat). For each value: mechanism (`Mechanism`), hive, key, name, type, value as written, the key's last write, and with `--binary` the fingerprints of the file it names. Every value is given, Windows' own included: telling the added from the default is the analysis's work. BITS jobs are not read yet.

`office_trust_records.json`
Office documents a user trusted ("Enable content"): application, version, document, date; `MacrosEnabled` when macros were enabled — the door of most macro infections.

`typed_paths.json`, `word_wheel_query.json`
paths typed in Explorer's address bar and terms searched there, per user and position (`MruPosition`, 0 = the most recent, as in `shellbags.json`).

`root_certificates.json`
certificates of the machine's and each user's root stores: subject, issuer, validity; a root added makes Windows trust whatever it signs (TLS interception, signed malware). `ThumbprintMismatch` flags a certificate whose SHA-1 is not its key's name.

`internet_settings.json`
each user's proxy and automatic configuration script.

`intune_logs.json`
the Intune Management Extension's logs (`ProgramData\Microsoft\IntuneManagement Extension\Logs`), on a machine managed by Microsoft Intune: the PowerShell scripts and applications the tenant pushed and ran as SYSTEM, their outcome, the device's

registration. One record per message (CMTrace format, on one line or several): `Message`, `Component`, `Type`, `Thread`, the date, written in local time, in UTC and in local time — exact when the log writes its offset (`BiasMinutes`).

`cloud_identity.json`
: the machine's join to Entra ID (SYSTEM's `Control\CloudDomainJoin`: `TenantId`, `UserEmail`, `IdpDomain`, the device certificate's thumbprint as `KeyName`, and the tenant's information, MDM enrolment URLs included) and the work accounts each user added (`WorkplaceJoin` of their NTUSER.DAT); every value as stored, `KeyLastWrite` dating the registration. The accounts' token caches (`TokenBroker`, `Microsoft.AAD.BrokerPlugin`) are collected raw, encrypted by DPAPI as Windows keeps them, never decrypted by WAC. Also collected raw, not converted: the ETW trace files Windows writes to its disk (`--etw_logs`), OneDrive's logs and settings (`--onedrive`, SharePoint list sync's excepted) and Defender for Endpoint's local folder (`--defender_endpoint`, without the sensor's binaries).

`firewall.json`
: firewall rules (name, action, direction, ports, program; every field in `Fields`) and the domain, private and public profile settings. On Windows XP SP2 and later and Server 2003 SP1, the exceptions of each profile — allowed programs and open ports — as rules of `Version` `XP`, with their `Profile` and `Scope`; XP SP1 has no firewall policy.

`firewall_log.json`
: the firewall's log (`pfirewall.log` and its `.old` rotation), where the `LogFilePath` of each profile puts it — firewall settings and group policy — and at its default place: one record per connection allowed or dropped, with `Action`, `Protocol`, `SourceAddress`, `Destination Address`, the ports, `Size`, `Direction` and, since Windows 10, `ProcessId`; the date, written in local time, given in UTC and in local time. It is the only record of network flows a Windows keeps on its disk; logging is off by default.

`installed_programs.json`
: installed programs, read in the `Uninstall` keys of the machine (64- and 32-bit) and of each user and Windows Installer products (`Scope` `MSI product`, with the cached package `Local Package`): all their values (name, publisher, version, folder, uninstall command, `Install Date` — the day only, as written) and the key's write time. A remote access tool installed, then uninstalled, often leaves its trace there.

`defender_configuration.json`
: Microsoft Defender's configuration read in the registry: the **exclusions** (paths, extensions, processes, addresses; `Source` `Settings` or `Policy`), which an intruder adds so that their tools are not scanned, and the values that set or switch off protection (real time, tamper protection, sample submission…).

`explorer_recent_docs.json`
: files each user opened, as Explorer keeps them (`RecentDocs`): by extension (`Extension`) and for every extension together, with their rank (`MruPosition`, 0 for the most recent), the shell item of the shortcut made in Recent, and the list's write time, which is when the file at position 0 was opened. The name outlives the file and the shortcut.

`mount_points.json`
: volumes and network shares each user's Explorer saw (`MountPoints2`): volume (GUID, to match with `mounted_device.json` for a USB key) or share (`Share`, `\\server\share`), the

entry's write time — the last time Explorer saw it —, its values, and its subkeys with their command (`Shell\AutoRun\command`: an old persistence).

`known_networks.json`
networks the machine joined (`NetworkList`): name, kind, category, gateway hardware address and DNS suffix of each signature, first and last connection. Windows writes these two dates in local time: WAC gives them in UTC with the offset in force at their date (measured against the `NetworkProfile` log). They tell where the machine has been.

`program_usage.json`
what each user launched or used, source by source (`Source`): `FeatureUsage` (launches, switches, taskbar clicks, per application, `Count`), `JumplistData` (last use, `LastUsed`), the Program Compatibility Assistant's store (programs run), compatibility layers (`Layers`), the programs RADAR diagnosed, the taskbar's pinned items (`Taskband`), each decoded as a whole shortcut, and the programs chosen to open each file type (`FileExts`: `OpenWithList` with `Extension` and `MruPosition`, 0 for the most recent; `UserChoice`, the default program). `OpenWithProgids` is not collected: installers fill it, the user has no part in it.

`devices.json`
devices the machine knew (`Source`): USB and SCSI (`Enum`), with their values and their install, first install, last arrival and last removal dates (`Install`, `FirstInstall`, `LastArrival`, `Last Removal`, UTC); portable devices (a phone, a camera); volume labels Windows Search kept per drive letter (`VolumeInfoCache`); disk, volume and portable-device interfaces (`Device Classes`), written when they first appeared.

`network_shares.json`
folders the machine shares (`LanmanServer\Shares`): name, path, settings as stored, and the decoded security descriptor — owner, and for each access control entry its type, mask, SID and name. The administrative shares (`C$`, `ADMIN$`) are not there: Windows does not store them.

`network_configuration.json`
network configuration read in the hives: host name and domain (`Global`), and for each adapter its description and driver, a forced hardware address (`NetworkAddress`), its current and permanent hardware address, its last IPv4 and IPv6 settings, and its DHCP lease (obtained and end, UTC). It stays readable once the machine is unplugged or the lease expired, which the live state no longer shows.

`event_log_configuration.json`
configuration of every event log read in the hives: the channels (`WINEVT\Channels`: `Enabled`, `Type`, `OwningPublisher`, access rights `ChannelAccess`, size and retention when set) and the classic logs (`Services\EventLog`: file, `MaxSize`, `Retention`, access) and the ETW sessions started at boot (`WMI\Autologger`: `Start`, file, size). An intruder who disables a channel, shrinks a log, opens its access or stops a session such as `EventLog-Security` leaves the trace there; `KeyLastWrite` dates the last change of each.

`boot_records.json`
what runs before Windows, read raw on each physical disk (the collection medium's disk excepted, unless it holds the system): its first mebibyte copied — the MBR's boot code, the partition table, the GPT header and entries, and the gap before the first partition, where a bootkit's code hides —, the first 16 sectors of each partition, and every file of the EFI

system partition, read by WAC's FAT reader (loaders, BCD). One record per disk: `PartitionStyle` (`MBR`, `GPT`, or `Volume` for a disk formatted whole, without a table, as a USB key: its first sector is the volume's, `BootSectorOemId`), `DiskId`, `MbrBootCodeSha256`, and each partition in sectors as the table stores it (`FirstSector`, `Sectors`), with its type, the SHA-256 of its first sectors, its boot sector's OEM name and, for the EFI partition, its files (`EfiFiles`: path, size, SHA-256). The logical partitions of an extended partition are not followed.

`fat_volumes.json`
FAT12, FAT16, FAT32 and exFAT volumes, the suspect's USB keys and cards included (the collection's volume excepted): every entry, present or **deleted** (`Deleted`), its path, 8.3 name, size, attributes and dates. A name deleted on FAT keeps its long name unless overwritten; otherwise its 8.3 name, first letter lost (`NameIncomplete`). Each date is given in UTC and in local time derived from UTC, as in every artefact: one instant reads the same everywhere. FAT stores the local time Windows computed when it wrote it: a modification earlier than the creation is a date kept by a copy, written at the copy, and converted with the offset of that moment (`ModifiedWrittenAtCopy`, `ModifiedOffsetApplied`, `ModifiedAsStored`) — otherwise a winter file copied in summer would be an hour off. exFAT keeps its offset (`ModifiedOffset`…), the UTC date is exact. The collection copies only the volume's metadata (boot sector, FAT, directories), not the files' content; with `--binary`, a program of these volumes cited by an artefact (run from a key, for instance) is fingerprinted and collected as on NTFS.

`shadow_copies.json`
shadow copies ("previous versions", restore points) of every fixed NTFS volume, **read raw** — the catalog and stores of the differential area, without the VSS service or its interface: WAC creates, mounts and exposes no copy. One object per copy, oldest first: `Volume`, `ShadowCopyId` (the identifier `vssadmin` shows), `ShadowCopySetId`, `StoreId`, `CreationTimeUtc`, `Context`, `Attributes`, `OperatingMachine`, `StoredBlocks` (16 KiB blocks whose earlier state the store keeps) and `ExhibitFolder`, the folder of the exhibit store its files go to (`vss\<instant>_<identifier>`). For each copy, the collection reads through it the hives (system, Amcache, the profiles of `\Users`) and the `.evtx` event logs; a file identical (SHA-256) to its newer version — on the volume as it is now, or in the next copy — is not copied again: the manifest records it as shared with that version. Every exhibit read through a copy carries `ShadowCopyId` in the manifest. A hive an intruder erased or changed, a cleared log, are found there as they were. Windows goes on writing while WAC reads: each file is read twice, the block lists read again before each reading, and done again if the two readings differ. Windows XP keeps no copy. A copy holds what was *on the disk* at its instant, and Windows writes the registry and the event logs late: a value or an event written less than about a minute before the copy may not be in it — Windows itself does not read them there (measured: absent 15 s after, present 90 s after).

`shadow_copy_events.json`
the events a copy's logs keep and the current logs no longer hold (a cleared log, records overwritten by rotation): the form of `events.json`, plus `ShadowCopyId` and `ShadowCopyCreationTimeUtc`, the newest copy that keeps it. Each event once, however many copies.

`shadow_copy_registry.json`
the entries of the registry artefacts (Run keys, services, accounts and groups, ShimCache, Amcache, BAM, UserAssist, MUICache, ShellBags, MRU, USB devices, installed programs, known networks…) converted from a copy's hives that the volume's conversion does not

give: a Run value, a service, an account deleted since, the trace of an erased tool. `Artefact` (the artefact's JSON file), `ShadowCopyId`, `ShadowCopyCreationTimeUtc` and `Entry`, the entry as the artefact gives it. Each entry once. An entry changed since is there too, as it was then. The copy is not converted whole: it would repeat, nearly identical, the volume's thousands of entries.

`srum.json`
resource usage (SRUM, `sru\SRUDB.dat`), one entry per row of every table: bytes sent and received per application and per interface (an exfiltration shows there, with the program), CPU cycles, bytes read and written, connections; 30 to 60 days. `Table` (the provider's GUID), `Provider` (documented name: `NetworkUsage`, `ApplicationResourceUsage`…), then the columns by name (letters and digits only), and `App`, `UserSID`, `UserName` resolved through `Sru DbIdMapTable`. The database is copied live: the last hour is in its logs `SRU*.log`, copied into the exhibit store for a replay (`esentutl /r`) on a copy.

`bits_jobs.json`
BITS jobs (`qmgr.db`): name, description, type, state, owner, **notification command line** (run when the transfer ends or fails: a persistence), delays, dates, and each file: source URL, destination, temporary file, sizes. The whole blob is given in hexadecimal: BITS 10 keeps more there (the creating program) at places not described. Before Windows 10, the jobs are in the queue files `qmgr0.dat` and `qmgr1.dat`, which BITS writes in turn: each job says which one it was read in (`QueueFile`), since the older one may keep an earlier state of the same job. On Vista (BITS 3.0), the delimiter of the jobs is known to no published tool; it was measured and is read the same way.

`wmi_persistence.json`
WMI event subscriptions, read **offline** in the CIM repository copied raw (`wbem\Repository`): no WMI query, nothing run on the machine. In *every* namespace, each filter (WQL query), consumer (command line, VBScript or JScript script, log, mail, and any class an intruder defined), binding and timer, with all its properties (`Properties`: name and value; `Creator SID` in text). A persistence invisible in the registry. The two dates at the start of an instance are not emitted: they are not creation dates. On Windows XP the repository is in `wbem\Repository\FS` and has another layout (MD5 hashes, 4-byte page entries, two mapping files in SP1): it is read the same way.

`web_client.json`
each user's browsing in the Chromium browsers (Edge, Chrome, Brave, Vivaldi, Chromium, Opera), read in the `History` database of each browser profile. One record per URL (`Type Url`: title, visit and typed counts, last visit), per visit (`Visit`: instant, `Transition` — `typed`, `link`, `reload`… — and its qualifiers, the visit it came from `FromUrl`, duration), per download (`Download`: whole redirection chain `UrlChain`, page and tab of origin, destination path, bytes received, state, opened, raw `DangerType`, SHA-256 when the browser computed it) and per term searched in the address bar (`SearchTerm`). Every record carries `Browser`, `BrowserProfile`, `SID` and `Source`. Chromium's times (microseconds since 1601, UTC) are converted exactly. The databases are read on a private copy with their journal, replayed as the browser would. Beside `History`, the exhibit store receives raw the profile's other databases (`Login Data`, `Web Data`, cookies, shortcuts, bookmarks, preferences) and `Local State`, whose DPAPI-protected key encrypts the saved passwords; never the caches. Firefox (`Browser Firefox`, each folder of `Profiles`) gives the same records from `places.sqlite` — visits with their type (`typed`, `link`, `download`…), downloads with

their destination, end and size —, plus `TypedInput` (what was typed in the address bar and the page reached) and, from `formhistory.sqlite`, `FormEntry`: each value typed in a form or the search bar (`FieldName searchbar-history`), with its dates. Copied raw beside them: cookies, `logins.json` and `key4.db` (saved logins, encrypted), extensions, preferences, session. Finally `TypedUrl`: each address typed in Internet Explorer or File Explorer's address bar (`TypedURLs` of the `NTUSER.DAT`, `MruPosition` 0 for the most recent) with the instant it was typed (`TypedURLsTime`, `TypedUtc`). `SavedLogin`: each saved login (Chromium's `Login Data`, Firefox's `logins.json`) — site, account (in clear in Chromium, encrypted and so absent in Firefox), creation, last use, use count, refusal to save (`Never Save`); `Cookie`: site, name, path, dates, flags. The passwords and the cookies' values, encrypted, are never decoded: the databases are in the exhibit store for an analysis that would hold the user's DPAPI key. What was *open*: `SessionEntry`, each page of each tab of the session files — Chromium's `Sessions\Session_*` and `Tabs_*`, Firefox's `sessionstore.jsonlz4` and its backups —, with `State` (`Open`, `Closed`, `ClosedTab`, `ClosedWindow`), the instant the page was shown (`NavigatedUtc`, Chromium, that of its visit in `History`) or the tab's last access (`TabLastAccessedUtc`, Firefox), the closing instant (`ClosedUtc`) and the page shown (`Shown`): a tab of a private window, or of a cleared history, may be found only there; `Favicon`, each visited page's icon, kept after the history is cleared; `AddressBar Prediction` (Chromium), what was typed in the address bar and the address then chosen. Firefox's bookmark backups, Internet Explorer's local folder and WinINet's containers (`INetCookies`, `IEDownloadHistory`) are copied raw. And `CacheEntry` (`Browser WebCache`): each entry of the `WebCacheV01.dat` database of Internet Explorer, Edge Legacy and the WinINet applications, by container (`Container`: `History`, `MSHist…`, `Content`, `iedownload`, `BackgroundTransferApi`) — URL, account and page of a history entry (`User`, `Page`), access count, dates, cache file (`CachedFile`: its path, through the subfolder the entry names). Internet Explorer's engine (WinINet) still serves `mshta.exe` and scripts even when IE is redirected to Edge: each profile's cache (`INetCache\IE`) is collected, limited to the types an investigation needs — HTA, scripts, executables, archives, documents with macros, PDF —, and the HTA an attack had run is found there with its entry. A download of Internet Explorer 10 and 11 (the `iedownload` container) also gives `Download`: its URL, MIME type, temporary copy in the cache, the path it was saved to (`TargetPath`, or only `Suggested Name` when it was not), its size and the server's address — a format Microsoft does not document, established on a real download by IE 11 in the test machines of Windows 11 and Windows 8.1, whose layouts differ; another layout is not guessed. On Windows 7 and earlier, Internet Explorer 5 to 9 kept the same data in `index.dat` files (cache, history and its `MSHist01` folders, cookies, downloads): they are collected and converted into `CacheEntry` (`Browser Internet Explorer`), recovered (`Recovered`) and dropped (`RecordKind LEAK`) entries included; a weekly history's visit time, local, is converted, the expiry and last check, FAT dates without a time zone, are given as written. The modification time of the `MSHist` containers, which Windows writes in local time, is converted to UTC with the offset in force at its date. The database is copied live with its logs: its last entries may be only there (`esentutl /r` replay by the analyst).

`dpapi_master_keys.json`

DPAPI master keys of the system (`System32\Microsoft\Protect`, `Scope System`) and of each profile (`AppData\Roaming\Microsoft\Protect`, `User`). The files are copied as they are into the exhibit store, still encrypted by the user's password or the system's DPAPI secret, with `Preferred` and `CREDHIST`: with that password, the analysis can later decrypt the saved

secrets (browser passwords, Wi-Fi, credential manager). The JSON gives their metadata only, never the key: owner (`SID`), `Guid` (the header's, checked equal to the file's name), version, policy, size of each block — a non-zero `DomainKeyLength`: a domain account —, the file's dates, and `Preferred` with `PreferredExpiresUtc` for the key in use.

`startup_folders.json`
content of the startup folders, of all users (`Scope AllUsers`) and of each profile (`User`, with `SID`): every file, its dates and fingerprints; a shortcut is decoded (`Shortcut`, as in `recentdocs.json`).

`shortcuts.json`
shortcuts put there to be clicked, outside the Recent folder: Start menu of all users and of each profile, desktop of each profile and public desktop, Quick Launch and items pinned to the task bar (`Location`: `StartMenu`, `Desktop`, `QuickLaunch`, `TaskBar`; `Scope` and `SID` as above). Each shortcut is decoded as in `recentdocs.json`: target, arguments, volume and machine of origin. An uninstalled program leaves its shortcut there; an intruder plants a command line there, or retargets an existing shortcut at their binary while keeping the icon. The Start menu's Startup folder is not in it: it belongs to `startup_folders.json`. A desktop redirected elsewhere on a local disk (OneDrive, another volume, see `user_shell_folders.json`) is read there too.

`user_shell_folders.json`
each user's folders (`UserShellFolders` of their `NTUSER.DAT`): name (`Name`, and `FolderName` for those a GUID names, as the Downloads), value as written (`Value`), resolved path (`Path`, absent for a network share) and last write of the key. OneDrive moves the desktop and the documents, a company redirects them to a share: that is where the user's files are to be looked for, and a redirection set quietly is itself a trace.

`powershell_history.json`
PSReadLine history of each profile and each host (`Host`: `ConsoleHost`, the ISE, VS Code): the commands in the order they were typed, a command on several lines joined. No date per command: `SourceModified` dates the last one. Looked for from Windows 7: there by default from Windows 10, it exists on 7, 8 and 8.1 once PowerShell 3 or later and PSReadLine were installed. Vista and XP stop at PowerShell 2, without a history on disk; their PowerShell logs are in `events.json`.

`powershell_profiles.json`
PowerShell profiles, the scripts PowerShell runs at each start — a persistence that goes through neither the registry, nor a task, nor a service. The machine's, for every user (Windows PowerShell's folder in `System32` and `SysWOW64`, PowerShell 6 and 7's in `ProgramFiles\PowerShell`), and each user's (`Documents\WindowsPowerShell` and `Documents\PowerShell`, where User Shell Folders puts the documents when they are moved): `Scope` (`Machine`, `User`), `Edition`, `Host` (`AllHosts` for `profile.ps1`, else the console, the ISE, VS Code), `Path`, dates, `Size`, `Sha256` and `Content`, the script's text. Looked for on every version: on XP and Vista, PowerShell is there only once installed.

`activities_cache.json`
Windows Timeline (`ActivitiesCache.db` of each account of each profile, with its `-wal` log): every row of the `Activity` and `ActivityOperation` tables (`Table`), all its columns. On Windows 10, the applications opened, the files and pages viewed (`Payload`: title, URI), the time spent in the foreground (`Payload UserEngaged`, from `StartTime` to `EndTime`) and,

when clipboard history is on, what was copied (`ClipboardPayload`); on Windows 11, which dropped the Timeline, only the synchronised settings. The dates are Unix times in UTC, given in UTC and in local time; the GUIDs in .NET's byte order; a zero date or GUID is not emitted. The codes (`ActivityType`, `ActivityStatus`, `OperationType`) are given as stored: Microsoft does not document them.

`search_index.json`

Windows Search index (Windows 11: `Windows.db` and `Windows-gather.db`, with their `-wal` logs). One record per indexed item of the property store (`Table PropertyStore`, `WorkId`): path (`ItemUrl`, and `ItemPathDisplay` in the interface's language), creation, modification, access and indexing dates, size, owner, type and, for a document, the start of its text (`SearchAutoSummary`); each property under its Windows name without the `System.` prefix. One record per document the crawler saw (`Table Gather`, `DocumentId`, the same number as `WorkId`): its URL and its last modification when crawled (on Vista, its path `ItemPath` in capitals, from the path table, and all its dates as FILETIMEs). The record outlives the file until the next crawl: a deleted document may still show there. Up to Windows 10, the index is an ESE database, `Windows.edb`: its logs are replayed into the working copy, then it is converted in the same form; a property's type, which that database does not say, comes from the table Windows 11 declares, and a property that table does not know is given in hexadecimal. Measured on Windows 8.1, and on Windows 7, whose database names its tables and columns otherwise, writes its numbers and dates big-endian and obfuscates its text; a value of Windows 10 compressed otherwise than on 7 bits (XPRESS) is said, not decoded. **Beware**: on Windows 7 and 8 the indexer keeps what it just indexed in memory for several minutes before writing it to its database; `WAC` reads the database as it is on disk — it restarts no service on the examined machine — and the last few minutes may be missing.

`rdp_client.json`

outgoing Remote Desktop connections of each profile: recent connections (`Recent Connection`, `MruPosition` 0 for the latest) and servers reached (`Server`: user name offered `UsernameHint`, hash of the certificate accepted `CertHash`, the key's last write), the `.rdp` files of the Documents (`ConnectionFile`, `Default.rdp` included, each setting), and the bitmap cache (`BitmapCache`): each tile of `Cache*.bin`, a piece of the remote screen as the client drew it, is exported as an image in `rdp_cache\<SID>\<file>`, with its size, keys and the SHA-256 of its pixels. The older `bcache22.bmc`, the client of Windows XP's, is decoded the same way: tiles compressed by RDP's interleaved RLE, 16 bits per pixel (`BitsPerPixel`), `UnreadTiles` counting those that could not be. The cache keeps what was shown on the screen, a text typed in clear in a visible field included. Another `.bmc`, of a depth WAC has no sample of, is counted without being decoded (`DecodingStatus UnsupportedFormat`).

`rdp_server.json`

incoming Remote Desktop connections, read in the collected logs: connection received and authentication (RemoteConnectionManager 261 and 1149), TCP connection accepted (RdpCoreTS 131), sessions (LocalSessionManager 21 to 25, 39 to 41), logons and logoffs of type 10 and reconnections (Security 4624, 4625, 4634, 4778, 4779). Each event keeps its data and is given `User`, `Domain`, `SourceAddress` and `SessionId`, whatever name its log gives them. Of little use on a workstation, it is the first question on a server.

`emails.json`

Outlook's mail of every profile: the PSTs (`Documents\Outlook Files`) and the OSTs, the

offline copy of an Exchange or Microsoft 365 account (`AppData\Local\Microsoft\Outlook`), read by WAC's own code — Unicode PSTs of Outlook 2003 and later, OSTs of Outlook 2013 and later (4 KiB pages, compressed blocks). A record per mailbox (kind, version, number of messages read, of unreadable ones) and one per message: folder, class, subject, sender, recipients (`Recipients`: name, address, To/Cc/Bcc), submission, delivery, creation and modification times, Internet message identifier, transport headers. The body (text, HTML) and every attachment are written next to the JSON (`emails\<SID>\<file>\<message>\`), with their size and SHA-256; an attachment's name, chosen by the sender, is never taken as a path: the file bears its rank and, when it is a plain one, its extension. An attached message (an appointment, a mail forwarded as attachment) gives its subject and class. The files Outlook opened from a message (`INetCache\Content.Outlook`) and the new Outlook's local data (`AppData\Local\Microsoft\Olk`) are collected as they are. Not decoded: the "high" (cyclic) encryption and the ANSI format of Outlook 97 to 2002, with no sample to check them on — the mailbox's record says so. Thunderbird: its profiles (`AppData\Roaming\Thunderbird\Profiles`) are collected whole — address book and accounts' settings included — and each mail folder, an mbox file, is decoded: headers (encoded words included), sending date in UTC, text and HTML bodies, attachments (base64, quoted-printable, RFC 2231 names), in the same fields as for Outlook, with `Offset` (the message's position in the file) instead of `NodeId`.

`sysmon.json`

Sysmon (Sysinternals), where it is installed: one record per installation, told by the link between the service (`Parameters\DriverName`) and its driver's key holding rules — found even renamed. The service and the driver with their `ImagePath`, the last write of `Parameters` (the last configuration change), every value of that key as stored (`Parameters`: `Rules`, `HashingAlgorithm`, `Options`, `ArchiveDirectory`, `ConfigFile`, `ConfigHash`), `RulesSha256`. Sysinternals does not document the rules' binary format: they are given in hexadecimal, not decoded. The archive folder (`ArchiveFolder`, `C:\Sysmon` by default) is taken whole: the files deleted under a `FileDelete` rule, named by their fingerprint — the very content of an erased tool —, listed in `ArchivedFiles` with their size and SHA-256; the XML configuration file, if still there, is taken and its SHA-256 given (`ConfigFileSha256`, to compare with `ConfigHash`). Sysmon's log is converted with the others into `events.json`, even moved; its version is there (event 4).

`thumbcache.json`

Explorer's thumbnail and icon caches of every profile (`thumbcache_*.db`, `iconcache_*.db`): what the user had on screen, even if the file was deleted since. For each entry: its cache and size, its key (`Hash`), its identifier, the `ThumbnailCacheId` Windows gives for the file, which ties a thumbnail to its file (the key from Windows 7 on, the identifier on Vista), its dimensions, its format (`png`, `jpg`, `bmp`), the SHA-256 of the image, and the image itself, exported next to the JSON (`thumbcache\<SID>\<cache>\<ThumbnailCacheId>.<ext>`). The two checksums Windows writes for each entry are recomputed (`HeaderChecksumValid`, `DataChecksumValid`): an entry read wrong fails them. Vista to Windows 11 (Vista and 7 give no dimensions; Vista gives the source file's extension, `SourceExtension`). Past the first free place the header gives, entries of an earlier state may remain whole: they are given as remnants (`Remnant`) when their header checksum holds, their image only when its own checksum holds too (`<ThumbnailCacheId>-remnant-<offset>.<ext>`); the reading stops at the first that does not. On Windows XP and Server 2003, the `Thumbs.db` that Explorer writes

in every folder shown as thumbnails, searched by name on every fixed NTFS volume: per thumbnail, the image's name and path, its modification when the thumbnail was made (`SourceModifiedUtc`), even after the image was deleted, the thumbnail's size and SHA-256, and its bytes exported as stored (`.bin`): a JPEG of four components without its tables, which no ordinary viewer renders.

`error_reports.json`
Windows error reports (`Report.wer`) of the machine and of each profile: program, path, instant (`EventTimeUtc`), problem signature (faulting module, version), every value of the report in `Values`.

`defender_log.json`
Microsoft Defender's support logs (`MPLog-*.log`): every line as written, dated when it starts with a UTC date. Run-time scans, detections, exclusions, updates. The WPP traces of the same folder (`MpWpp*.bin`, binary ETW traces only Microsoft's private symbols decode) are copied raw into the exhibit store, not converted.

`defender_detections.json`
one entry per DetectionHistory file, which survives the clearing of the event log: threat, resource (file, key), account, process, initial detection date, quarantine entry, and the named `ThreatTracking` properties (fingerprints, size, start time). The format is undocumented: only the fields whose position was confirmed against `MSFT_MpThreatDetection` are named; the others are given as read (`UnnamedFields`: offset, type, value), without a meaning made up. Defender of Windows 8.1 writes the same fields without the `MagicVersion` marker of Windows 10 and 11: they are named the same way, confirmed the same way.

`defender_quarantine.json`
quarantine entries: threat, instant of the quarantine, original path of the removed file, its creation and modification dates, attributes, size, and the fingerprint of its content (`ContentSha256`). The quarantine is copied **encrypted**, as Defender keeps it: nothing malicious is written in clear, on the collection medium or on the analysis workstation. The content is decrypted in memory only, for its fingerprint.

`hosts.json`
entries of the `hosts` file: address and names, line number; an untouched Windows has none. A file past 16 MiB is not read (logged): it stays in the exhibit store.

6.5.3 Files and browsing

`mft.json`
every file and directory of every fixed NTFS volume (`Volume`: `C:`, `G:`…; a record number only means something within its volume), present or **deleted** (`Deleted`) as long as NTFS has not reused its record: rebuilt path, size, attributes, and two sets of four UTC dates — `CreatedUtc`… (the `$STANDARD_INFORMATION` attribute, what Explorer shows and what a timestomping tool rewrites) and `FileNameCreatedUtc`… (the `$FILE_NAME` attribute, written by NTFS alone). A `$STANDARD_INFORMATION` date earlier than the `$FILE_NAME` one betrays timestomping. `PathIncomplete` marks a path one of whose parent directories was reused since: the path is not made up. A file with hard links (`HardLinks`) is one record under

several names — `System32\crypt32.dll` and its original under `WinSxS`: `HardLinkPaths` gives its other paths, each one searchable. `SecurityId` is the number of its security descriptor in `security_descriptors.json`.

`security_descriptors.json`
security descriptors of every fixed NTFS volume (`$Secure:$SDS`): each once, however many files share it, identified by `Volume` and `SecurityId` — the key every file of `mft.json` carries. It gives the **owner** (`Owner`, `OwnerName`), the group, and each entry of the DACL and SACL (type, flags, access mask, SID and name). An intruder's tool dropped by their account, a system file no longer owned by TrustedInstaller, an ACL opened to Everyone are read there, deleted files included. NTFS stores with each descriptor a hash of its bytes: WAC recomputes it, and `HashMismatch` flags a descriptor that does not match it.

`deleted_files.json`
the **content** of the deleted files, where `mft.json` gives only their record: the tool, the script or the archive an intruder removed stays in its clusters until NTFS gives them to another file. Read raw, nothing opened nor written on the examined machine. A resident content (a few hundred bytes, in the record itself) is given as it is; each cluster of a non-resident one is checked free in `$Bitmap` before and after it is read: given meanwhile to another file, it becomes a hole of zeros. A cluster taken then given back since the deletion reads free: the runs of every deleted record of the `$MFT` are therefore crossed, and a cluster a later deleted file held becomes a hole (`Reason` "clusters claimed since by a later deleted file"). An intermediate file whose record was reused leaves no runs: what it wrote is not seen this way. `State`: `Recovered` (the whole content), `Partial` (holes listed in `MissingRanges`, first byte and length), `Overwritten` (nothing left), `Zeroed` (clusters read back zero: a solid-state drive that received TRIM), `NotRecoverable` and `Reason` (compressed, encrypted by EFS, runs partly in an extension record, inconsistent runs), `RecordReused` (the record went to another file since the copy of the `$MFT`), `NotRead` (not read: `max_total_mib` reached). With the path (`Path`), the record's dates, the deletion's time (`DeletedUtc`, read in the change journal: no record carries it), `RecoveredBytes` and the fingerprints `Recovered SHA256`… of the content *as recovered*, holes included: the original file's only when the state is `Recovered`. A copied content lies in the exhibit store under `X:\$DeletedFiles\<record>-<sequence>_<name>` (`Exhibit`), never under its former path, where another file may be. Since Windows 10, a file deleted while still open is first renamed into `$Extend\$Deleted`: its record names it there, and `FormerPath` gives the path it had, read in the change journal. A recovered executable follows the rule of the others (`binary`): an authentic one is listed with its `Signature`, not copied (`Copied false`, `NotCopied`); under `binary: none` nothing is judged, and it is copied. The most recent first (the record's latest date, the deletion's not being in it), up to `max_total_mib`; the `excluded_folders` (Windows updates) are not read, nor the output folder. The setting is a structure under `artefacts:`; `deleted_files: false` is enough to leave it out. `Source` says where the entry comes from: `MftRecord`, a deleted record of the `$MFT` (level 1); `IndexSlack` (level 2), a name left in the slack of a directory's index block (`$I30`) or in a block the directory released: the name, the directory (`Path`), the file's record (`Entry`, `Sequence`), its size and the four dates of its `$FILE_NAME` (`FileNameCreated Utc`…), `State NameOnly` — no content. Only a name no directory holds any more is given, and not one level 1 already gives. `CurrentPath`: the same file still exists at this path, moved or renamed, not deleted. `Carved` (level 3, `level: 3` only: the whole free space is read, for as long as it takes): a file carved from the free clusters level 1 did not give, without a name or

a date — `Format` (`PE`, `ZIP` and the Office documents, `PDF`), first cluster (`Lcn`), `Size`. Only these formats, whose structure gives their exact size and can be checked, are carved; a file is taken only if all its clusters are contiguous, free and not given by level 1 — a fragmented file is not found, never cut wrong. Exhibit under `X:\$CarvedFiles\<cluster>.<ext>`. `FatEntry`: a deleted entry of a FAT or exFAT volume (a suspect's key): its first cluster (`FirstCluster`), its size, its dates as stored (`ModifiedAsStored`, the local time of the machine that wrote it, or `ModifiedUtc` when exFAT gives its offset). The FAT chain is erased: the content is read as the clusters that follow the first, each checked free — `State` `RecoveredIfContiguous` and `ContiguityAssumed`, unless exFAT knew the file unfragmented (`Recovered`). Level 3 carves the free space of these volumes too. On a BitLocker volume, the raw reading sees the volume as the system decrypts it, as for the other artefacts: its free clusters too.

`usn_journal.json`
: the change journal of every fixed NTFS volume (`$UsnJrnl:$J`, `Volume`; the `Usn` numbers rise within a volume): every creation, write, rename and **deletion**, dated, with its reasons (`Reasons`, an unknown bit kept in hexadecimal) and the path rebuilt through the `$MFT`. It keeps the trace of files dropped then erased, which the `$MFT` has already forgotten. The `$MFT`, the change journal, `$LogFile` (the transaction journal, not converted) and `$TxfLog` (the journal of transactional NTFS, a CLFS log, its containers and the `$Tops:$T` stream, not converted) are also copied raw into the exhibit store, to rebuild the timeline later with another tool.

`ntfs_volumes.json`
: every fixed NTFS volume, read in its boot sector (`$Boot`, recognised by its signatures before any field): serial number (`VolumeSerial`, the 32 bits shortcuts, jump lists and Prefetch files record of their target's volume, and `VolumeSerial64` whole), sector, cluster, record and index block sizes, place of the `$MFT` and of its mirror. Then the change journal's header (`$UsnJrnl:$Max`, `UsnJournal`): maximum size, allocation delta, `UsnJournalId`, which is the journal's **creation** date (`Created`), and the first number still present. A journal deleted and recreated (`fsutil usn deletejournal`) shows a creation far later than the installation.

`recentdocs.json`
: recent-document shortcuts: target, timestamps of the shortcut and of the target, target size, volume (type, serial number, label), network path, shell item list, and the extra data blocks: the tracker block (`Tracker`: NetBIOS name of the machine the target was on, tracking identifiers, and what their version-1 GUID reveals — when the tracking began and the machine's hardware address), paths with environment variables, known folder, target properties.

`jumplistAutomaticDestinations.json`, `jumplistCustomDestinations.json`
: per-application jump lists, with the shortcuts they hold; in the automatic ones, each shortcut comes with its DestList entry (last access, machine name, tracking identifiers with their date and hardware address, pin status, number of openings `InteractionCount`, rank `MruPosition` — 0 for the most recent —, name of the known folder targeted); in the custom ones, their categories in order (`Categories`: a custom one and its name, a known one — frequent or recent —, the tasks).

`shellbags.json`
: folders browsed in Explorer, as a rebuilt tree; for each folder, its rank among its siblings (`MruPosition`, 0 for the most recent), the write time of the key holding it (`LastWriteTime`)

and of its own key (`KeyLastWriteTime`: its addition, or the browsing of a subfolder), and its `NodeSlot`. The "first" and "last interaction" of other tools are only readings of these two times. On Windows XP, Explorer's folders are in NTUSER.DAT (`ShellNoRoam\BagMRU`), read the same way.

`mrus.json`, `mruApps.json`
most-recently-used files and applications. On Windows XP, as text: the file's path (`Path`), the program and its folder (`Folder`), ranked by `ID`, 0 for the most recent.

`zone_identifiers.json`
`Zone.Identifier` streams ("Mark of the Web") that browsers and mail clients write on what they download: file, zone (`ZoneId`, `ZoneName` — `Internet` for 3), page of origin (`ReferrerUrl`) and the file's address (`HostUrl`), every value in `Values`. Read in the copy of the `$MFT`, where this small stream is resident: no further read of the examined machine; a non-resident stream, rare, is flagged (`ContentNotResident`).

`recycle_bin.json`
files sent to the recycle bin, on every fixed NTFS volume, from their `$I` index: original path, size, instant of the deletion (`DeletedUtc`), and the user (`SID`). Only the `$I` are collected; the files themselves (`$R`) are in `mft.json`. On Windows XP and Server 2003, from `INFO2` of `RECYCLER\<SID>`, one entry per record, with the copy's name in the bin (`RecycledName`, `D<drive><n>.<ext>`, `Dc1.txt` for a file of C:); the size is the space the file occupied.

6.5.4 Devices

`setupapi.json`
sections of `setupapi.dev.log` and its rotations: operation (`Operation`), device or driver (`Subject`, with a USB key's serial number), start and end converted from the examined machine's local time, outcome. The first connection of a device is dated there, even when the `USBSTOR` key was cleaned. On Windows XP, from `setupapi.log`: a section per dated line, its first message as the operation, the device an `#I121` line names as the subject, `SUCCESS` by that line or the first error as the outcome; no end date.

`Usbstor.json`
USB storage devices known to the system: `Vendor`, `Product`, `Revision` and `Type`, from `DeviceId`; `SerialNumber`, from `InstanceId` — absent when Windows made the identifier up (second character "&"), the device declaring none; first and last insertion dates.

`mounted_device.json`
mapping between drive letters and devices. `Drive` names the letter or the volume; `Type` gives the form of the value: `GPT partition` (`PartitionGuid`), `MBR partition` (`DiskSignature` and `PartitionOffset`, in bytes), `Device path` (`Device`, the device path, with the serial number of a USB key) or `Unknown` (`Data`, the bytes in hexadecimal).

6.5.5 Event logs (`--events`)

`events.json` holds one object per event, in the same form whatever the log's format: the `.evt` logs of Windows XP and Server 2003 are read into the fields Windows renders for a

classic source in an `.evtx` (identifier and its qualifiers, level from the event type, category as the task, strings as `Data`, binary data as `Binary`), their circular area walked from the oldest record. A log the machine's registry places outside its folder (the `File` value of its channel, `wevtutil sl /lfn`, or of its classic log) — moved for room, or by an intruder so that nobody looks for it — is taken where it is and converted with the others:

`EvtSystem*`
: the system section: provider, identifier, level, task, keywords, creation time, record number, process and thread, channel, computer, user;

`EvtEventData`
: the event's data, as a list of `Name` / `Value` pairs;

`EvtEventMessage`
: the readable message, as Event Viewer would show it, rebuilt from the provider's resources collected on the machine: manifest (`WEVT_TEMPLATE`, or its twin `SystemResources\*.mun` on Windows 11) and message table in the machine's interface language, or a classic source's `EventMessageFile` files. Each value is shown as Windows shows it: label of its value map or of its flags, hexadecimal, text of a Win32, HRESULT or NTSTATUS error code, network address;

`EvtSourceLog`
: the log file the event comes from. One channel can be carried by several files (current log and archives) whose record numbers overlap;

`EvtRecovered`
: `true` for an event of a *cleared* log: Windows resets the chunk header without wiping the records, which WAC reads again. Event Viewer no longer shows them; they remain the trace of what the clearing meant to erase, and must never be read as live events.

> **Incomplete messages**
>
> A `%1`, `%2`… mark left in a message means the event does not carry the expected data: it is kept visible so that the sentence does not look complete. Not every provider ships message resources: their events then have no `EvtEventMessage`, but their data are complete.

CHAPTER 7

Reading the data

7.1 Timestamps

Every date is emitted twice: `X` in local time and `XUtc` in UTC. The local time is the **suspect's**, computed with the time zone read from its `SYSTEM` hive — not the collecting machine's. Dates carry the fraction of a second when the source provides it, and only to that source's precision: seven digits for a `FILETIME`, three for a date to the millisecond, none for a date to the second (FAT date, precise to two seconds, text date, Unix time).

Every date carries the offset in force **at that date**, daylight saving time included: a January date collected in summer in Paris is at `+01:00`, a July date at `+02:00`. WAC applies the rules of the examined machine year by year (`Dynamic DST` key of the `SOFTWARE` hive): a 2005 date in the United States follows the rules of 2005. The local time written is always derived from the UTC time: one instant reads the same in every artefact. Two dates have no knowable time zone and are given **as written**, without offset nor UTC, with `TimeZone` saying so: the date of an item seen inside a ZIP archive (`ModifiedAsWritten`; a ZIP entry keeps the clock time of the machine that made the archive — the archives Windows 11 reads itself, 7z or tar, hold a UTC instant instead, given as such) and an OLE date of a property store (`AsWritten`). Placing them on a timeline needs their origin: WAC does not make it up. For a value the artefact stores in local time (FAT volume, `setupapi.dev.log`), the hour repeated in autumn is read as its first occurrence, and the hour skipped in spring with the offset before the change, as Windows does.

7.2 Account names

The fields that name an account (`SIDName`, `Owner`, `RunAs`) come from the evidence alone, without asking the system or the network: local accounts and groups of the SAM database (in the language of the examined machine), well-known SIDs under their canonical names (`SYSTEM`, `LOCAL SERVICE`…), service SIDs (`NT SERVICE\…`), and the profile folder for a domain account. A SID none of them names is published without a name: the SID remains the evidence, for the analyst to resolve against the domain.

The Default profile. `C:\Users\Default` is the template copied for every new profile: a Run value, a shortcut or an association put there reaches every account created afterwards. WAC reads it as a profile (`NTUSER.DAT` and `UsrClass.dat` hives, desktop, Start menu, Recent…). Belonging to no account, it has no SID: its entries carry the SID `S-0-0-0`, zeros only, in a SID's form so that tools link them, but no SID at all (revision 0), and `Profile`: `Default` instead of `SIDName`. It is not `S-1-0-0`, Windows' real NULL SID.

7.3 GUID names

The fields that name a GUID (COM class, known folder, handler of a task) come first from a reference table, then from the examined machine's hives: its classes (`SOFTWARE\Classes\CLSID`), its known folders (`FolderDescriptions`), and each user's classes (`UsrClass.dat`). A component installed on the machine is therefore named, with the module that serves it. A GUID nothing names is published without a name.

7.4 A missing field is not an empty field

A value with no source is not emitted at all. An empty key would read as a failed read, and could not be told apart from "the hive does not hold this value". What could not be read is recorded in `investigation.json`.

7.5 Artefact not collected

When a whole artefact could not be read, its file says so explicitly (the text is produced in French):

```
{
  "Artifact": "USBSTOR (registre)",
  "CollectionStatus": "NotCollected",
  "Error": "0x02 Le fichier spécifié est introuvable.",
  "Note": "La lecture de cet artefact a échoué : l'absence de données
           ci-dessus ne signifie PAS qu'aucune trace n'existe sur le système."
}
```

> **Warning**
> An empty artefact file and a "NotCollected" file do not say the same thing. The first means no entry was found; the second, that the source could not be read.

7.6 Artefact absent from the version of Windows

An artefact does not exist on every version of Windows: no SRUM before Windows 8, no jump list before Windows 7. WAC reads the examined machine's version in its `SOFTWARE` hive, announces it at the start of the collection and records it in `investigation.json`. An artefact asked for that this version does not have is neither launched nor counted as a failure, and its file says so:

```
{
  "Artifact": "RESOURCE USAGE (SRUM)",
  "CollectionStatus": "AbsentFromThisVersion",
  "WindowsRelease": "Windows 7",
  "WindowsBuild": 7601,
```

```
  "Note": "This artefact does not exist on this version of Windows
           (artefacts.srum): there was nothing to collect. Its absence is
           normal and says nothing about what happened on the system."
}
```

A version newer than those WAC knows is treated as the last one known, and flagged as such: the collection goes on. The table below gives, for each key of `wac.yml`, the first Windows that has the artefact, where it lies and its format according to the version. It is produced from the program's own table, the only one WAC consults. Version 2 of WAC is only validated under Windows 10 and 11; the older forms are listed for what comes next.

The locations are written as Windows Vista and later lay out their folders. On Windows XP and Server 2003, WAC looks for them where those versions keep the same data: `AppData\Local` becomes `LocalSettings\ApplicationData`, `AppData\Roaming` becomes `ApplicationData`, the recent documents and the Start menu are at the profile's root, `Documents` becomes `MyDocuments`, and `ProgramData` becomes `AllUsers\ApplicationData`, next to the profiles.

Artefact	From	Where	Format	Converted
`activities_cache`	Windows 10	`Users\<user>\AppData\Local\ConnectedDevicesPlatform\<account>\ActivitiesCache.db`	SQLite	yes
	Still written by Windows 11 (measured, build 26200), though the Timeline is no longer shown. Server editions to be measured.			
`amcache`	Windows 7	`Windows\AppCompat\Programs\RecentFileCache.bcf`	BCF	yes
	Windows 8	`Windows\AppCompat\Programs\Amcache.hve`	registry hive, File and Programs keys	yes
	Windows 10	`Windows\AppCompat\Programs\Amcache.hve`	registry hive, Inventory* keys	yes
	always there from Windows 10; Amcache.hve also on Windows 7 with update KB2952664.			
`bam`	Windows 10	`SYSTEM\CurrentControlSet\Services\bam\UserSettings`	registry values, FILETIME	yes
	Windows 10	`SYSTEM\CurrentControlSet\Services\bam\State\UserSettings`	registry values, FILETIME	yes
	Server editions to be measured.			
`bits`	Windows XP	`ProgramData\Microsoft\Network\Downloader\qmgr0.dat, qmgr1.dat`	BITS state file	yes
	Windows 10	`ProgramData\Microsoft\Network\Downloader\qmgr.db`	ESE	yes
	Before Vista: Documents and Settings\All Users\Application Data\Microsoft\Network\Downloader. The queue files measured on Windows 8.1; the earlier versions read as bits_parser (ANSSI) describes them, not measured.			

Artefact	From	Where	Format	Converted
`boot_records`	Windows XP	`each physical disk: its first mebibyte, each partition's first 16 sectors; the EFI system partition's files`	MBR, GPT, boot sectors; FAT	yes
	The EFI system partition exists only on a machine started by UEFI (Windows 8 and later, some 7 and Vista).			
`cloud_identity`	Windows 10	`SYSTEM\CurrentControlSet\Control\CloudDomainJoin, NTUSER.DAT\…\WorkplaceJoin; TokenBroker, AAD.Broker Plugin of each profile`	registry keys; token caches copied encrypted	yes
	Present only on a machine joined to Entra ID or with a work account added.			
`defender`	Windows 8	`ProgramData\Microsoft\Windows Defender\Support\MPLog-*.log`	UTF-16 text	yes
	Windows 10	`ProgramData\Microsoft\Windows Defender\Scans\History\Service\DetectionHistory`	DetectionHistory records	yes
	Server 2012 and 2012 R2 have no built-in Defender; quarantine copied encrypted on every version.			
`defender_configuration`	Windows Vista	`SOFTWARE\Microsoft\Windows Defender, SOFTWARE\Policies\Microsoft\Windows Defender`	registry values	yes
`defender_endpoint`	Windows 10	`ProgramData\Microsoft\Windows Defender Advanced Threat Protection`	files, raw copy	no
	Present only where Defender for Endpoint is onboarded; its formats are not documented: copied raw, not converted.			
`deleted_files`	Windows XP	`$MFT, $Bitmap, $I30, free clusters; FAT and exFAT volumes`	NTFS 3.1 file records, data runs, index blocks; FAT directory entries; PE, ZIP, PDF carved	yes
	What a deleted file still holds depends on the disk: a solid-state drive that received TRIM returns zeros for the freed clusters, declared as such, never as the content.			
`devices`	Windows XP	`SYSTEM\CurrentControlSet\Enum\USB, SCSI`	registry keys	yes
	Windows Vista	`…and SOFTWARE\Microsoft\Windows Portable Devices, device properties with dates (Windows 8)`	registry keys	yes

Artefact	From	Where	Format	Converted
dpapi_master_keys	Windows XP	<profile>\Application Data\Microsoft\Protect\<SID>	master key file, version 2	yes
	Windows Vista	Users\<user>\AppData\Roaming\Microsoft\Protect\<SID>	master key file, version 2	yes
emails	Windows XP	Outlook PST and OST, Thunderbird mbox, in each profile	PST/OST (ANSI and Unicode), mbox	yes
	Depends on the mail program installed, not on Windows.			
error_reports	Windows Vista	ProgramData\Microsoft\Windows\WER, Users\<user>\AppData\Local\Microsoft\Windows\WER	Report.wer, UTF-16 text	yes
	Windows XP: Dr Watson (drwtsn32.log, user.dmp), not collected.			
etw_logs	Windows Vista	Windows\System32\LogFiles\WMI, Windows\System32\WDI, Windows\Logs, ProgramData\Microsoft\Diagnosis\ETLLogs	ETL trace files, raw copy	no
	Copied raw: not converted (kernel events in MOF form, providers' manifests).			
event_log_configuration	Windows XP	SYSTEM\CurrentControlSet\Services\EventLog	registry keys	yes
	Windows Vista	…and SOFTWARE\Microsoft\Windows\CurrentVersion\WINEVT\Channels, Control\WMI\Autologger	registry keys	yes
	always there from Windows XP			
events	Windows XP	Windows\System32\config*.evt	EVT (LfLe records)	yes
	Windows Vista	Windows\System32\winevt\Logs*.evtx	EVTX, binary XML	yes
	always there from Windows XP			
explorer_recent_docs	Windows XP	NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs	MRUListEx and shell items	yes
fat_volumes	Windows XP	FAT12, FAT16, FAT32 volumes	FAT directory entries	yes
	Windows Vista	…and exFAT volumes (Vista SP1; XP with KB955704)	exFAT directory entries	yes
firewall	Windows XP	SYSTEM\CurrentControlSet\Services\SharedAccess\Parameters\FirewallPolicy (XP SP2)	registry values, profiles and port lists	yes

Artefact	From	Where	Format	Converted
	Windows Vista	`…\FirewallPolicy\FirewallRules`	registry values, rule strings v2.x	yes
`firewall_log`	Windows XP	`Windows\pfirewall.log (XP SP2), or the profile's LogFilePath`	W3C text, local time	yes
	Windows Vista	`Windows\System32\LogFiles\Firewall\pfirewall.log, or the profile's LogFilePath`	W3C text, local time; process since Windows 10	yes
	Off by default: present only where the firewall's logging was switched on.			
`hosts`	Windows XP	`Windows\System32\drivers\etc\hosts`	text	yes
	always there from Windows XP			
`installed_programs`	Windows XP	`SOFTWARE and NTUSER.DAT …\CurrentVersion\Uninstall, Installer\UserData`	registry keys	yes
	always there from Windows XP			
`internet_settings`	Windows XP	`NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Internet Settings`	registry values	yes
`intune_logs`	Windows 10	`ProgramData\Microsoft\IntuneManagementExtension\Logs\*.log`	CMTrace text, local time	yes
	Present only on a machine managed by Microsoft Intune, once the management extension was installed.			
`jump_lists`	Windows 7	`Users\<user>\AppData\Roaming\Microsoft\Windows\Recent\AutomaticDestinations, CustomDestinations`	OLE compound file, DestList version 1	yes
	Windows 10	`the same folders`	OLE compound file, DestList version 3 and 4	yes
	Windows 11	`the same folders`	OLE compound file, DestList version 6	yes
	DestList version 6 measured on build 26200; the custom lists (CustomDestinations) carry version 2 in their header, any other is not read.			
`known_networks`	Windows Vista	`SOFTWARE\Microsoft\Windows NT\CurrentVersion\NetworkList`	registry keys, SYSTEMTIME dates	yes
`local_groups`	Windows XP	`SAM\SAM\Domains\Builtin\Aliases`	SAM alias records	yes
	always there from Windows XP			

Artefact	From	Where	Format	Converted
log_file	Windows XP	$LogFile	NTFS log, raw copy	no
	Windows Vista	…and $Extend\$RmMetadata\$TxfLog	transactional NTFS log, raw copy	no
	Copied raw on every version: not converted.			
mft	Windows XP	$MFT	NTFS 3.1 file records	yes
mounted_devices	Windows XP	SYSTEM\MountedDevices	registry values	yes
	always there from Windows XP			
mount_points	Windows XP	NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Explorer\MountPoints2	registry keys	yes
mru	Windows XP	NTUSER.DAT …\Explorer\ComDlg32\OpenSaveMRU, LastVisitedMRU	MRUList and strings	yes
	Windows Vista	NTUSER.DAT …\Explorer\ComDlg32\OpenSavePidlMRU, LastVisitedPidlMRU	MRUListEx and shell items	yes
mru_apps	Windows XP	NTUSER.DAT …\Explorer\ComDlg32\LastVisitedMRU	MRUList and strings	yes
	Windows Vista	NTUSER.DAT …\Explorer\ComDlg32\LastVisitedPidlMRU	MRUListEx and shell items	yes
muicache	Windows XP	NTUSER.DAT\Software\Microsoft\Windows\ShellNoRoam\MUICache	registry values	yes
	Windows Vista	UsrClass.dat\Local Settings\Software\Microsoft\Windows\Shell\MuiCache	registry values	yes
network	Windows XP	live: TCP/IP tables of the running system	observation	yes
network_configuration	Windows XP	SYSTEM\CurrentControlSet\Services\Tcpip\Parameters\Interfaces, Control\Network	registry values	yes
	Windows 10	…and Control\NetworkSetup2\Interfaces\<guid>\Kernel: hardware addresses	registry values	yes
	always there from Windows XP; Before Windows 10, the adapters' hardware addresses are in no documented key of the hives: not given.			
network_shares	Windows XP	SYSTEM\CurrentControlSet\Services\LanmanServer\Shares	registry values, security descriptors	yes

Artefact	From	Where	Format	Converted
ntfs_volumes	Windows XP	$Boot, $Extend\$UsnJrnl:$Max	NTFS boot sector, journal header	yes
office_trust_records	Windows XP	NTUSER.DAT\Software\Microsoft\Office\<version>\<program>\Security\Trusted Documents\TrustRecords	registry values	yes
	Depends on Office (2010 and later), not on Windows.			
onedrive	Windows 7	Users\<user>\AppData\Local\Microsoft\OneDrive\logs and settings	ODL logs, sync database, raw copy	no
	Present only where OneDrive ran; SharePoint list sync's logs (ListSync) left out. Copied raw: not converted.			
operating_system	Windows XP	SOFTWARE\Microsoft\Windows NT\CurrentVersion, SYSTEM\…\Control	registry values	yes
	always there from Windows XP			
persistence	Windows XP	SOFTWARE, SYSTEM and NTUSER.DAT: IFEO, AppInit_DLLs, Winlogon, LSA, Active Setup…	registry values	yes
	always there from Windows XP			
powershell_history	Windows 7	Users\<user>\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt	UTF-8 text	yes
	Written by PSReadLine: by default from Windows 10; on Windows 7, 8 and 8.1 once PowerShell 3 or later and PSReadLine are installed. Vista and XP stop at PowerShell 2, which PSReadLine does not run on.			
powershell_profiles	Windows XP	Windows\System32\Windows PowerShell\v1.0*profile.ps1, Windows\SysWOW64\WindowsPowerShell\v1.0, Users\<user>\Documents\WindowsPowerShell	PowerShell script	yes
	Windows 7	…and Program Files\Power Shell\<version>, Users\<user>\Documents\PowerShell	PowerShell script	yes
	Windows XP and Vista: only once PowerShell is installed, absent by default. A user's documents also where User Shell Folders moves them.			
prefetch	Windows XP	Windows\Prefetch*.pf	SCCA version 17	yes
	Windows Vista	Windows\Prefetch*.pf	SCCA version 23	yes
	Windows 8	Windows\Prefetch*.pf	SCCA version 26	yes

Artefact	From	Where	Format	Converted
	Windows 10	`Windows\Prefetch\*.pf`	SCCA version 30 and 31, compressed (MAM)	yes
	Switched off by default on the server editions.			
`processes`	Windows XP	`live: processes of the running system`	observation	yes
`program_usage`	Windows XP	`NTUSER.DAT …\AppCompatFlags\Layers`	registry values	yes
	Windows Vista	`…and AppCompatFlags\Compatibility Assistant\Store, Explorer\Taskband`	registry values	yes
	Windows 10	`…and Explorer\FeatureUsage, Search\JumplistData`	registry values	yes
`rdp_client`	Windows XP	`NTUSER.DAT\Software\Microsoft\Terminal Server Client; …\Terminal Server Client\Cache\bcache*.bmc`	registry values; BMC bitmap cache	yes
	Windows 7	`…and Cache\Cache????.bin`	BIN bitmap cache	yes
	bcache22.bmc decoded (16 bits per pixel, measured on XP SP1); another .bmc is counted, its tiles not exported.			
`rdp_server`	Windows Vista	`Security.evtx, Terminal Services-LocalSessionManager and RemoteConnectionManager logs`	EVTX events	yes
	Windows XP and Server 2003: events 528 and 682 of Security.evt, not converted.			
`recent_documents`	Windows XP	`<profile>\Recent\*.lnk`	shell link	yes
	Windows Vista	`Users\<user>\AppData\Roaming\Microsoft\Windows\Recent\*.lnk`	shell link	yes
`recycle_bin`	Windows XP	`RECYCLER\<SID>\INFO2`	INFO2 records, version 5	yes
	Windows Vista	`$Recycle.Bin\<SID>\$I*`	$I version 1	yes
	Windows 10	`$Recycle.Bin\<SID>\$I*`	$I version 2	yes
`registry_backups`	Windows XP	`Windows\repair`	registry hives	no
	Windows Vista	`Windows\System32\config\RegBack`	registry hives, raw copy	no
	Empty since Windows 10 1803 unless the backup task was switched back on. Copied raw: not converted.			

Artefact	From	Where	Format	Con-verted
`root_certificates`	Windows XP	`SOFTWARE and NTUSER.DAT …\SystemCertificates\Root, AuthRoot`	certificate blobs	yes
	always there from Windows XP			
`run_keys`	Windows XP	`SOFTWARE and NTUSER.DAT …\CurrentVersion\Run, RunOnce`	registry values	yes
	always there from Windows XP			
`scheduled_tasks`	Windows XP	`Windows\Tasks\*.job`	JOB binary	yes
	Windows Vista	`Windows\System32\Tasks, SOFTWARE …\Schedule\Task Cache`	XML, registry values	yes
	always there from Windows Vista			
`search_index`	Windows Vista	`ProgramData\Microsoft\Search\Data\Applications\Windows\Windows.edb`	ESE	yes
	Windows 11	`…\Windows\Windows.db, Windows-gather.db`	SQLite	yes
	Windows.edb converted with its 7-bit text (Windows 8.1 measured), and Windows 7's layout: its own table and column names, big-endian numbers, obfuscated text (measured); a value compressed otherwise (XPRESS) is said, not decoded. Windows XP: Windows Desktop Search, an optional install.			
`security`	Windows XP	`SECURITY hive: Policy\Secrets, Cache, PolAdtEv`	registry values	yes
	always there from Windows XP			
`security_descriptors`	Windows XP	`$Secure:$SDS`	NTFS security descriptor stream	yes
`services`	Windows XP	`SYSTEM\CurrentControlSet\Services`	registry keys	yes
	always there from Windows XP			
`sessions`	Windows XP	`live: logon sessions of the running system`	observation	yes
`setupapi`	Windows XP	`Windows\setupapi.log`	text, XP layout	yes
	Windows Vista	`Windows\INF\setupapi.dev.log`	text, sections	yes
	always there from Windows XP			
`shadow_copies`	Windows Vista	`System Volume Information\{3808876b-c176-4e48-b7ae-04046e6cc752}, {GUID}{3808876b-…}`	VSS catalog and stores, 16 KiB blocks	yes

Artefact	From	Where	Format	Converted
	Windows XP keeps no persistent copy: its restore points are folders of their own. Server 2003 catalogs no store (no entry of type 3): its copies are listed, not read.			
`shellbags`	Windows XP	`NTUSER.DAT\Software\Microsoft\Windows\Shell and ShellNoRoam`	BagMRU shell items	yes
	Windows Vista	`UsrClass.dat\Local Settings\Software\Microsoft\Windows\Shell, and NTUSER.DAT`	BagMRU shell items	yes
`shimcache`	Windows XP	`SYSTEM\…\Control\Session Manager\AppCompatibility\AppCompatCache`	header 0xDEADBEEF, 96 entries	yes
	Windows XP	`SYSTEM\…\Control\Session Manager\AppCompatCache\AppCompatCache`	header 0xBADC0FFE, the file's size	no
	Windows Vista	`the same value`	header 0xBADC0FFE, entries of 24 or 32 bytes	yes
	Windows 7	`the same value`	header 0xBADC0FEE	yes
	Windows 8	`the same value`	header 0x80, entries "00ts"	yes
	Windows 8.1	`the same value`	header of 0x80 bytes starting with 0, entries "10ts"	yes
	Windows 10	`the same value`	header 0x30 or 0x34, entries "10ts"	yes
	always there from Windows XP; Windows 8 and 8.1 have a header of 0x80 bytes: they are told apart by the tag of their first entry, which is their signature here. Windows 8.1 measured on build 9600. Server 2003 and Vista share their magic: the version picks the form; Vista measured on SP2 x86, Server 2003 not yet.			
`shortcuts`	Windows XP	`Start menu, desktops, Quick Launch of each profile and of All Users`	shell link	yes
`srum`	Windows 8	`Windows\System32\sru\SRUDB.dat`	ESE	yes
	always there from Windows 8			
`startup_folders`	Windows XP	`Start Menu\Programs\Startup of each profile and of All Users`	files	yes
`sysmon`	Windows 7	`SYSTEM\CurrentControlSet\Services\<Sysmon driver>\Parameters, its Archive Directory`	registry keys; Rules as stored	yes

Artefact	From	Where	Format	Converted
	Not part of Windows: present only where Sysmon was installed. Its rules' binary format is not documented: given as stored, with its fingerprint.			
`thumbcache`	Windows XP	`Thumbs.db in each folder`	OLE compound file	yes
	Windows Vista	`Users\<user>\AppData\Local\Microsoft\Windows\Explorer\thumbcache_*.db`	CMMM, Vista layout	yes
	Windows 7	`the same folder`	CMMM, Windows 7 layout	yes
	Windows 8	`the same folder, with iconcache_*.db`	CMMM, Windows 8 and later layout	yes
`typed_paths`	Windows Vista	`NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Explorer\TypedPaths`	registry values	yes
`usbstor`	Windows XP	`SYSTEM\CurrentControlSet\Enum\USBSTOR`	registry keys	yes
	Windows 8	`…with the device properties' dates (first install, last connection, last removal)`	registry keys, FILETIME	yes
`userassist`	Windows XP	`NTUSER.DAT …\Explorer\UserAssist\{5E6AB780-…}, {75048700-…}`	ROT13 names, 16-byte values	yes
	Windows 7	`NTUSER.DAT …\Explorer\UserAssist\{CEBFF5CD-…}, {F4E57C4B-…}`	ROT13 names, 72-byte values	yes
`users`	Windows XP	`SAM\SAM\Domains\Account\Users`	F and V records, hashes under RC4 (boot key)	yes
	Windows 10	`the same keys`	F and V records, hashes under AES	yes
	always there from Windows XP			
`user_shell_folders`	Windows XP	`NTUSER.DAT …\Explorer\User Shell Folders`	registry values	yes
`usn_journal`	Windows XP	`$Extend\$UsnJrnl:$J`	USN_RECORD_V2	yes
	Windows 8	`the same stream`	USN_RECORD_V2, V3 (128-bit references)	yes
	Windows 8.1	`the same stream`	USN_RECORD_V2, V3, V4 (range tracking)	yes
	The journal exists only where it was created: always on the system volume from Vista on.			

Artefact	From	Where	Format	Converted
`web_client`	Windows XP	`Internet Explorer 5 to 9: index.dat of History, cache, cookies, downloads`	MSIE Cache File 5.2	yes
	Windows 8	`Internet Explorer 10 and 11, Edge Legacy: …\WebCache\WebCacheV01.dat`	ESE	yes
	Windows 10	`…and Chromium browsers, Firefox: History, places.sqlite, session files`	SQLite, SNSS, mozLz4	yes
	Chromium and Firefox depend on the browser installed, not on Windows.			
`wmi`	Windows XP	`Windows\System32\wbem\Repository\FS`	CIM repository of Windows XP (INDEX.BTR, OBJECTS.DATA, MAPPING, in FS)	yes
	Windows Vista	`Windows\System32\wbem\Repository`	CIM repository (INDEX.BTR, OBJECTS.DATA, MAPPING)	yes
	always there from Windows XP; Windows XP keeps the repository in a FS subfolder, MD5 hashes and 4-byte page entries (measured, SP1 and SP3).			
`word_wheel_query`	Windows 7	`NTUSER.DAT …\Explorer\WordWheelQuery`	MRUListEx and UTF-16 strings	yes
	Windows XP: Search Assistant\ACMru, not collected.			
`zone_identifiers`	Windows XP	`<file>:Zone.Identifier (XP SP2)`	alternate data stream, INI text	yes

One line per form of each artefact: the first Windows that has it, where it lies, its format, and whether WAC converts that form (a form not converted is collected raw at most).

7.7 Coverage of MITRE ATT&CK

ATT&CK describes, technique by technique, how to detect it: analytics, each naming the sources it needs — a data component (“Process Creation”) and the log source that carries it (“WinEventLog:Sysmon EventCode=1”). It is a list kept outside WAC, against which its coverage is measured. The tables below are produced from it: first what WAC does with each log source, then the artefacts that keep on the disk a trace of each data component beyond the logs, last, for each Windows technique, whether a log WAC takes or an artefact on the disk holds a trace of it, and the sources it does not take, with their reason. A trace held is not a detection: looking for it in the JSON files belongs to the analysis.

Log source	WAC
`wineventlog:sysmon…`	`events`, `sysmon`
`wineventlog:iis…`	server role (IIS, AD FS, domain controller): version 4.0
`wineventlog:microsoft-iis…`	server role (IIS, AD FS, domain controller): version 4.0
`wineventlog:adfs…`	server role (IIS, AD FS, domain controller): version 4.0
`wineventlog:…`	`events`
`autoruns:registryscan…`	`persistence`, `run_keys`, `services`, `scheduled_tasks`, `startup_folders`
`windows firewall log…`	`firewall_log`
`applicationlog:intune…`	`intune_logs`, `events`
`applicationlog:iis…`	server role (IIS, AD FS, domain controller): version 4.0
`applicationlog:callrecords…`	cloud service: in the tenant's logs; the machine's side is taken (cloud_identity, onedrive, intune_logs)
`etw…`	live telemetry (ETW, API calls, performance counters): read by a consumer as it happens; the trace files Windows writes to its disk are taken raw (etw_logs)
`windows:perfmon…`	live telemetry (ETW, API calls, performance counters): read by a consumer as it happens; the trace files Windows writes to its disk are taken raw (etw_logs)
`windows:osquery…`	live telemetry (ETW, API calls, performance counters): read by a consumer as it happens; the trace files Windows writes to its disk are taken raw (etw_logs)
`edr…`	an EDR product's own telemetry: in its console; Defender for Endpoint's local folder is taken raw (defender_endpoint)
`m365defender…`	an EDR product's own telemetry: in its console; Defender for Endpoint's local folder is taken raw (defender_endpoint)
`m365:defender…`	an EDR product's own telemetry: in its console; Defender for Endpoint's local folder is taken raw (defender_endpoint)
`m365:…`	cloud service: in the tenant's logs; the machine's side is taken (cloud_identity, onedrive, intune_logs)
`azure:…`	cloud service: in the tenant's logs; the machine's side is taken (cloud_identity, onedrive, intune_logs)
`aws:…`	cloud service: in the tenant's logs; the machine's side is taken (cloud_identity, onedrive, intune_logs)
`nsm:…`	network capture: on the wire, not on the machine
`network traffic…`	network capture: on the wire, not on the machine
`dns:query…`	network capture: on the wire, not on the machine
`firmware:…`	firmware: not read, WAC reads the disk

Data component	On the disk, beyond the logs
Process Creation	`prefetch`, `amcache`, `shimcache`, `bam`, `userassist`, `muicache`, `srum`, `program_usage`
Process Access	—
Process Modification	—
Process Metadata	`processes`
Process Termination	—
Command Execution	`powershell_history`, `prefetch`
Script Execution	`powershell_history`, `powershell_profiles`
OS API Execution	—
Module Load	—
File Creation	`mft`, `usn_journal`, `zone_identifiers`
File Modification	`mft`, `usn_journal`, `log_file`
File Deletion	`usn_journal`, `deleted_files`, `recycle_bin`
File Access	`jump_lists`, `recent_documents`, `shortcuts`, `shellbags`, `explorer_recent_docs`
File Metadata	`mft`, `security_descriptors`, `zone_identifiers`
Network Connection Creation	`srum`, `network`, `firewall_log`, `rdp_client`, `bits`, `web_client`
Network Traffic Content	—
Network Traffic Flow	`srum`, `firewall_log`
Network Share Access	`mount_points`, `network_shares`, `shellbags`
Windows Registry Key Modification	`run_keys`, `persistence`, `services`, `registry_backups`, `shadow_copies`
Windows Registry Key Creation	`run_keys`, `persistence`, `services`, `registry_backups`, `shadow_copies`
Windows Registry Key Access	—
Service Creation	`services`
Service Modification	`services`
Service Metadata	`services`
Driver Load	`services`
Scheduled Job Creation	`scheduled_tasks`
Scheduled Job Modification	`scheduled_tasks`
Scheduled Job Metadata	`scheduled_tasks`
WMI Creation	`wmi`
Logon Session Creation	`users`, `rdp_server`
Logon Session Metadata	`users`, `sessions`, `rdp_server`
User Account Authentication	`users`
User Account Creation	`users`
User Account Modification	`users`, `local_groups`
User Account Deletion	—
User Account Metadata	`users`, `local_groups`, `security`
Group Enumeration	—
Firewall Rule Modification	`firewall`

Data component	On the disk, beyond the logs
Drive Creation	`usbstor`, `mounted_devices`, `devices`, `setupapi`, `mount_points`
Drive Access	`mount_points`, `shellbags`, `fat_volumes`
Drive Modification	`ntfs_volumes`, `boot_records`
Volume Creation	`mounted_devices`, `ntfs_volumes`, `shadow_copies`
Host Status	—
Named Pipe Metadata	—
Application Log Content	`defender`, `setupapi`, `error_reports`, `intune_logs`, `onedrive`
Web Credential Creation	`web_client`, `dpapi_master_keys`, `cloud_identity`
Firmware Modification	—
Active Directory Object Modification	—
Active Directory Object Access	—
Active Directory Object Creation	—
Active Directory Object Deletion	—
Active Directory Credential Request	—
Cloud Service Metadata	`cloud_identity`
Cloud Storage Access	`onedrive`
Instance Creation	—
Instance Start	—
Domain Registration	—

ID	Technique	Logs	Disk	Sources not taken
T1001	Data Obfuscation	—	—	NSM
T1001.001	Junk Data	yes	yes	NSM
T1001.002	Steganography	yes	yes	NSM
T1001.003	Protocol or Service Impersonation	yes	yes	NSM
T1003	OS Credential Dumping	yes	yes	
T1003.001	LSASS Memory	yes	yes	
T1003.002	Security Account Manager	yes	yes	
T1003.003	NTDS	yes	yes	
T1003.004	LSA Secrets	yes	yes	
T1003.005	Cached Domain Credentials	yes	yes	
T1003.006	DCSync	yes	—	NSM
T1005	Data from Local System	yes	yes	
T1006	Direct Volume Access	yes	yes	
T1007	System Service Discovery	yes	yes	
T1008	Fallback Channels	yes	yes	NSM

ID	Technique	Logs	Disk	Sources not taken
T1010	Application Window Discovery	yes	yes	
T1011	Exfiltration Over Other Network Medium	yes	yes	
T1011.001	Exfiltration Over Bluetooth	yes	yes	
T1012	Query Registry	yes	yes	
T1014	Rootkit	yes	yes	
T1016	System Network Configuration Discovery	yes	yes	
T1016.001	Internet Connection Discovery	yes	yes	
T1016.002	Wi-Fi Discovery	yes	yes	
T1018	Remote System Discovery	yes	yes	
T1020	Automated Exfiltration	yes	yes	
T1021	Remote Services	yes	yes	
T1021.001	Remote Desktop Protocol	yes	yes	
T1021.002	SMB/Windows Admin Shares	yes	yes	
T1021.003	Distributed Component Object Model	yes	yes	
T1021.004	SSH*	yes	—	
T1021.005	VNC	yes	yes	NSM
T1021.006	Windows Remote Management	yes	yes	NSM
T1025	Data from Removable Media	yes	yes	
T1027	Obfuscated Files or Information	yes	yes	
T1027.001	Binary Padding	yes	yes	
T1027.002	Software Packing	yes	yes	
T1027.003	Steganography	yes	yes	
T1027.004	Compile After Delivery	yes	yes	
T1027.005	Indicator Removal from Tools	yes	yes	
T1027.006	HTML Smuggling	yes	yes	EDR, NSM
T1027.007	Dynamic API Resolution	yes	yes	ETW
T1027.008	Stripped Payloads	yes	yes	EDR
T1027.009	Embedded Payloads	yes	yes	EDR
T1027.010	Command Obfuscation	yes	yes	
T1027.011	Fileless Storage	yes	yes	
T1027.012	LNK Icon Smuggling	yes	yes	
T1027.013	Encrypted/Encoded File	yes	yes	
T1027.014	Polymorphic Code	yes	yes	
T1027.015	Compression	yes	yes	
T1027.016	Junk Code Insertion	yes	yes	
T1027.017	SVG Smuggling	yes	yes	
T1027.018	Invisible Unicode	yes	yes	
T1029	Scheduled Transfer	yes	yes	
T1030	Data Transfer Size Limits	yes	yes	NSM
T1033	System Owner/User Discovery	yes	yes	

ID	Technique	Logs	Disk	Sources not taken
T1036	Masquerading	yes	yes	
T1036.001	Invalid Code Signature	yes	yes	
T1036.002	Right-to-Left Override	yes	yes	
T1036.003	Rename Legitimate Utilities	yes	yes	EDR
T1036.004	Masquerade Task or Service	yes	yes	
T1036.005	Match Legitimate Resource Name or Location	yes	yes	
T1036.007	Double File Extension	yes	yes	
T1036.008	Masquerade File Type	yes	yes	
T1036.010	Masquerade Account Name	yes	yes	ETW
T1036.012	Browser Fingerprint	yes	yes	ETW, NSM
T1037	Boot or Logon Initialization Scripts	yes	yes	
T1037.001	Logon Script (Windows)	yes	yes	
T1037.003	Network Logon Script	yes	yes	
T1039	Data from Network Shared Drive	yes	yes	
T1040	Network Sniffing	yes	yes	
T1041	Exfiltration Over C2 Channel	yes	yes	NSM
T1046	Network Service Discovery	yes	yes	
T1047	Windows Management Instrumentation	yes	yes	
T1048	Exfiltration Over Alternative Protocol	yes	yes	
T1048.001	Exfiltration Over Symmetric Encrypted Non-C2 Protocol	yes	yes	
T1048.002	Exfiltration Over Asymmetric Encrypted Non-C2 Protocol	yes	yes	NSM
T1048.003	Exfiltration Over Unencrypted Non-C2 Protocol	yes	yes	NSM
T1049	System Network Connections Discovery	yes	yes	
T1052	Exfiltration Over Physical Medium	yes	yes	
T1052.001	Exfiltration over USB	yes	yes	
T1053	Scheduled Task/Job	yes	yes	
T1053.002	At	yes	yes	
T1053.005	Scheduled Task	yes	yes	
T1055	Process Injection	yes	yes	ETW
T1055.001	Dynamic-link Library Injection	yes	yes	
T1055.002	Portable Executable Injection	yes	yes	
T1055.003	Thread Execution Hijacking	yes	yes	ETW
T1055.004	Asynchronous Procedure Call	yes	yes	ETW
T1055.005	Thread Local Storage	yes	—	EDR
T1055.011	Extra Window Memory Injection	yes	yes	ETW
T1055.012	Process Hollowing	yes	yes	ETW

ID	Technique	Logs	Disk	Sources not taken
T1055.013	Process Doppelgänging	yes	yes	ETW
T1055.015	ListPlanting	yes	yes	ETW
T1056	Input Capture	yes	yes	
T1056.001	Keylogging	yes	yes	
T1056.002	GUI Input Capture	yes	yes	
T1056.003	Web Portal Capture	yes	yes	4.0
T1056.004	Credential API Hooking	yes	—	
T1057	Process Discovery	yes	yes	
T1059	Command and Scripting Interpreter	yes	yes	
T1059.001	PowerShell	yes	yes	
T1059.003	Windows Command Shell	yes	yes	EDR
T1059.005	Visual Basic	yes	yes	
T1059.006	Python	yes	yes	EDR
T1059.007	JavaScript	yes	yes	EDR
T1059.010	AutoHotKey & AutoIT	yes	yes	
T1059.011	Lua	yes	yes	
T1068	Exploitation for Privilege Escalation	yes	yes	
T1069	Permission Groups Discovery	yes	yes	
T1069.001	Local Groups	yes	yes	
T1069.002	Domain Groups	yes	yes	
T1070	Indicator Removal	yes	yes	
T1070.003	Clear Command History	yes	yes	
T1070.004	File Deletion	yes	yes	
T1070.005	Network Share Connection Removal	yes	yes	NSM
T1070.006	Timestomp	yes	yes	EDR
T1070.007	Clear Network Connection History and Configurations	yes	yes	EDR
T1070.008	Clear Mailbox Data	yes	yes	cloud
T1070.009	Clear Persistence	yes	yes	
T1070.010	Relocate Malware	yes	yes	
T1071	Application Layer Protocol	yes	yes	NSM
T1071.001	Web Protocols	yes	yes	NSM
T1071.002	File Transfer Protocols	yes	yes	NSM
T1071.003	Mail Protocols	yes	yes	NSM
T1071.004	DNS	yes	yes	NSM
T1071.005	Publish/Subscribe Protocols	yes	yes	NSM
T1072	Software Deployment Tools	yes	yes	
T1074	Data Staged	yes	yes	
T1074.001	Local Data Staging	yes	yes	
T1074.002	Remote Data Staging	yes	yes	
T1078	Valid Accounts	yes	yes	

ID	Technique	Logs	Disk	Sources not taken
T1078.001	Default Accounts	yes	yes	
T1078.002	Domain Accounts	yes	yes	
T1078.003	Local Accounts	yes	yes	
T1080	Taint Shared Content	yes	yes	
T1082	System Information Discovery	yes	yes	
T1083	File and Directory Discovery	yes	yes	
T1087	Account Discovery	yes	yes	
T1087.001	Local Account	yes	yes	
T1087.002	Domain Account	yes	yes	NSM
T1087.003	Email Account	yes	yes	
T1090	Proxy	yes	yes	NSM
T1090.001	Internal Proxy	yes	yes	
T1090.002	External Proxy	yes	yes	
T1090.003	Multi-hop Proxy	yes	yes	NSM
T1090.004	Domain Fronting	yes	yes	NSM
T1091	Replication Through Removable Media	yes	yes	
T1092	Communication Through Removable Media	yes	yes	
T1095	Non-Application Layer Protocol	yes	yes	NSM
T1098	Account Manipulation	yes	yes	
T1098.002	Additional Email Delegate Permissions	yes	yes	cloud
T1098.005	Device Registration	yes	yes	
T1098.007	Additional Local or Domain Groups	yes	yes	
T1102	Web Service	yes	yes	NSM
T1102.001	Dead Drop Resolver	yes	yes	ETW
T1102.002	Bidirectional Communication	yes	yes	ETW
T1102.003	One-Way Communication	yes	yes	ETW
T1104	Multi-Stage Channels	yes	yes	
T1105	Ingress Tool Transfer	yes	yes	
T1106	Native API	yes	yes	
T1110	Brute Force	yes	yes	
T1110.001	Password Guessing	yes	yes	
T1110.002	Password Cracking	yes	yes	
T1110.003	Password Spraying	yes	yes	
T1110.004	Credential Stuffing	yes	yes	
T1111	Multi-Factor Authentication Interception	yes	yes	
T1112	Modify Registry	yes	yes	
T1113	Screen Capture	yes	yes	
T1114	Email Collection	yes	yes	

ID	Technique	Logs	Disk	Sources not taken
T1114.001	Local Email Collection	yes	yes	
T1114.002	Remote Email Collection	yes	yes	cloud
T1114.003	Email Forwarding Rule	yes	yes	cloud
T1115	Clipboard Data	yes	yes	
T1119	Automated Collection	yes	yes	
T1120	Peripheral Device Discovery	yes	yes	
T1123	Audio Capture	yes	yes	
T1124	System Time Discovery	yes	yes	EDR, ETW
T1125	Video Capture	yes	yes	
T1127	Trusted Developer Utilities Proxy Execution	yes	yes	
T1127.001	MSBuild	yes	yes	EDR
T1127.002	ClickOnce	yes	yes	
T1127.003	JamPlus	yes	yes	
T1129	Shared Modules	yes	yes	
T1132	Data Encoding	yes	yes	NSM
T1132.001	Standard Encoding	yes	yes	EDR
T1132.002	Non-Standard Encoding	yes	yes	EDR
T1133	External Remote Services	yes	yes	
T1134	Access Token Manipulation	yes	yes	ETW
T1134.001	Token Impersonation/Theft	yes	yes	ETW
T1134.002	Create Process with Token	yes	yes	ETW
T1134.003	Make and Impersonate Token	yes	yes	ETW
T1134.004	Parent PID Spoofing	yes	yes	ETW
T1134.005	SID-History Injection	yes	yes	ETW
T1135	Network Share Discovery	yes	yes	ETW
T1136	Create Account	yes	yes	
T1136.001	Local Account	yes	yes	
T1136.002	Domain Account	yes	yes	
T1137	Office Application Startup	yes	yes	
T1137.001	Office Template Macros	yes	yes	
T1137.002	Office Test	yes	yes	
T1137.003	Outlook Forms	yes	yes	
T1137.004	Outlook Home Page	yes	yes	
T1137.005	Outlook Rules	yes	yes	
T1137.006	Add-ins	yes	yes	
T1140	Deobfuscate/Decode Files or Information	yes	yes	
T1176	Software Extensions	yes	yes	
T1176.001	Browser Extensions	yes	yes	
T1176.002	IDE Extensions	yes	yes	
T1185	Browser Session Hijacking	yes	yes	

ID	Technique	Logs	Disk	Sources not taken
T1187	Forced Authentication	yes	yes	NSM
T1189	Drive-by Compromise	yes	yes	ETW, NSM
T1190	Exploit Public-Facing Application	yes	yes	4.0
T1195	Supply Chain Compromise	yes	yes	NSM
T1195.001	Compromise Software Dependencies and Development Tools	yes	yes	NSM
T1195.002	Compromise Software Supply Chain	yes	yes	NSM
T1195.003	Compromise Hardware Supply Chain	yes	yes	
T1197	BITS Jobs	yes	yes	
T1199	Trusted Relationship	yes	yes	
T1200	Hardware Additions	yes	yes	
T1201	Password Policy Discovery	yes	yes	
T1202	Indirect Command Execution	yes	yes	
T1203	Exploitation for Client Execution	yes	yes	
T1204	User Execution	yes	yes	
T1204.001	Malicious Link	yes	yes	NSM
T1204.002	Malicious File	yes	yes	
T1204.004	Malicious Copy and Paste	yes	yes	NSM
T1204.005	Malicious Library	yes	yes	
T1205	Traffic Signaling	yes	yes	
T1205.001	Port Knocking	yes	yes	
T1205.002	Socket Filters	yes	yes	NSM
T1207	Rogue Domain Controller	yes	—	cloud, NSM
T1210	Exploitation of Remote Services	yes	yes	NSM
T1211	Exploitation for Stealth	yes	yes	
T1212	Exploitation for Credential Access	yes	yes	
T1213	Data from Information Repositories	yes	yes	cloud
T1213.002	Sharepoint	—	yes	cloud
T1213.006	Databases	yes	yes	
T1216	System Script Proxy Execution	yes	yes	
T1216.001	PubPrn	yes	yes	
T1216.002	SyncAppvPublishingServer	yes	yes	
T1217	Browser Information Discovery	yes	yes	
T1218	System Binary Proxy Execution	yes	yes	
T1218.001	Compiled HTML File	yes	yes	
T1218.002	Control Panel	yes	yes	
T1218.003	CMSTP	yes	yes	
T1218.004	InstallUtil	yes	yes	
T1218.005	Mshta	yes	yes	
T1218.007	Msiexec	yes	yes	
T1218.008	Odbcconf	yes	yes	

ID	Technique	Logs	Disk	Sources not taken
T1218.009	Regsvcs/Regasm	yes	yes	
T1218.010	Regsvr32	yes	yes	
T1218.011	Rundll32	yes	yes	
T1218.012	Verclsid	yes	yes	
T1218.013	Mavinject	yes	yes	
T1218.014	MMC	yes	yes	
T1218.015	Electron Applications	yes	yes	
T1219	Remote Access Tools	yes	yes	
T1219.001	IDE Tunneling	yes	yes	NSM
T1219.002	Remote Desktop Software	yes	yes	
T1219.003	Remote Access Hardware	yes	yes	
T1220	XSL Script Processing	yes	yes	
T1221	Template Injection	yes	yes	
T1222	File and Directory Permissions Modification	yes	yes	
T1222.001	Windows Permissions	yes	yes	
T1480	Execution Guardrails	yes	yes	
T1480.001	Environmental Keying	yes	yes	
T1480.002	Mutual Exclusion	yes	yes	
T1482	Domain Trust Discovery	yes	yes	
T1484	Domain or Tenant Policy Modification	yes	yes	
T1484.001	Group Policy Modification	yes	yes	
T1484.002	Trust Modification	yes	yes	
T1485	Data Destruction	yes	yes	
T1486	Data Encrypted for Impact	yes	yes	
T1489	Service Stop	yes	yes	
T1490	Inhibit System Recovery	yes	yes	
T1491	Defacement	yes	yes	
T1491.001	Internal Defacement	yes	yes	
T1491.002	External Defacement	yes	yes	NSM
T1495	Firmware Corruption	yes	yes	
T1496	Resource Hijacking	yes	yes	ETW
T1496.001	Compute Hijacking	yes	yes	
T1496.002	Bandwidth Hijacking	yes	yes	
T1497	Virtualization/Sandbox Evasion	yes	yes	
T1497.001	System Checks	yes	yes	
T1497.002	User Activity Based Checks	yes	yes	
T1497.003	Time Based Checks	yes	yes	
T1498	Network Denial of Service	yes	yes	
T1498.001	Direct Network Flood	yes	yes	
T1498.002	Reflection Amplification	yes	yes	ETW

ID	Technique	Logs	Disk	Sources not taken
T1499	Endpoint Denial of Service	yes	yes	
T1499.001	OS Exhaustion Flood	yes	yes	NSM
T1499.002	Service Exhaustion Flood	yes	yes	ETW
T1499.003	Application Exhaustion Flood	yes	yes	ETW
T1499.004	Application or System Exploitation	yes	yes	
T1505	Server Software Component	yes	yes	
T1505.001	SQL Stored Procedures	yes	yes	
T1505.002	Transport Agent	yes	yes	
T1505.003	Web Shell	yes	yes	NSM
T1505.004	IIS Components	yes	yes	4.0
T1505.005	Terminal Services DLL	yes	yes	
T1518	Software Discovery	yes	yes	
T1518.001	Security Software Discovery	yes	yes	
T1518.002	Backup Software Discovery	yes	yes	
T1529	System Shutdown/Reboot	yes	yes	
T1531	Account Access Removal	yes	yes	
T1534	Internal Spearphishing	yes	yes	cloud
T1539	Steal Web Session Cookie	yes	yes	
T1542	Pre-OS Boot	yes	yes	
T1542.001	System Firmware	yes	yes	
T1542.002	Component Firmware	yes	yes	firmware
T1542.003	Bootkit	yes	yes	
T1543	Create or Modify System Process	yes	yes	
T1543.003	Windows Service	yes	yes	
T1546	Event Triggered Execution	yes	yes	
T1546.001	Change Default File Association	yes	yes	
T1546.002	Screensaver	yes	yes	
T1546.003	Windows Management Instrumentation Event Subscription	yes	yes	
T1546.007	Netsh Helper DLL	yes	yes	
T1546.008	Accessibility Features	yes	yes	
T1546.009	AppCert DLLs	yes	yes	
T1546.010	AppInit DLLs	yes	yes	
T1546.011	Application Shimming	yes	yes	
T1546.012	Image File Execution Options Injection	yes	yes	
T1546.013	PowerShell Profile	yes	yes	
T1546.015	Component Object Model Hijacking	yes	yes	
T1546.016	Installer Packages	yes	yes	
T1546.018	Python Startup Hooks*	—	yes	
T1547	Boot or Logon Autostart Execution	yes	yes	
T1547.001	Registry Run Keys / Startup Folder	yes	yes	

ID	Technique	Logs	Disk	Sources not taken
T1547.002	Authentication Package	yes	yes	
T1547.003	Time Providers	yes	yes	
T1547.004	Winlogon Helper DLL	yes	yes	
T1547.005	Security Support Provider	yes	yes	
T1547.008	LSASS Driver	yes	yes	
T1547.009	Shortcut Modification	yes	yes	
T1547.010	Port Monitors	yes	yes	
T1547.012	Print Processors	yes	yes	
T1547.014	Active Setup	yes	yes	
T1548	Abuse Elevation Control Mechanism	yes	yes	
T1548.002	Bypass User Account Control	yes	yes	
T1550	Use Alternate Authentication Material	yes	yes	
T1550.002	Pass the Hash	yes	yes	
T1550.003	Pass the Ticket	yes	yes	
T1552	Unsecured Credentials	yes	yes	
T1552.001	Credentials In Files	yes	yes	
T1552.002	Credentials in Registry	yes	yes	EDR
T1552.003	Shell History*	—	yes	
T1552.004	Private Keys	yes	yes	
T1552.006	Group Policy Preferences	yes	yes	
T1553	Subvert Trust Controls	yes	yes	
T1553.002	Code Signing	yes	yes	
T1553.003	SIP and Trust Provider Hijacking	yes	yes	
T1553.004	Install Root Certificate	yes	yes	
T1553.005	Mark-of-the-Web Bypass	yes	yes	
T1553.006	Code Signing Policy Modification	yes	yes	
T1554	Compromise Host Software Binary	yes	yes	
T1555	Credentials from Password Stores	yes	yes	
T1555.003	Credentials from Web Browsers	yes	yes	
T1555.004	Windows Credential Manager	yes	yes	
T1555.005	Password Managers	yes	yes	
T1556	Modify Authentication Process	yes	yes	
T1556.001	Domain Controller Authentication	yes	yes	
T1556.002	Password Filter DLL	yes	yes	
T1556.005	Reversible Encryption	yes	yes	
T1556.006	Multi-Factor Authentication	yes	yes	
T1556.007	Hybrid Identity	yes	yes	
T1556.008	Network Provider DLL	yes	yes	
T1557	Adversary-in-the-Middle	yes	yes	
T1557.001	Name Resolution Poisoning and SMB Relay	yes	yes	NSM

ID	Technique	Logs	Disk	Sources not taken
T1557.002	ARP Cache Poisoning	yes	yes	
T1557.003	DHCP Spoofing	yes	yes	NSM
T1558	Steal or Forge Kerberos Tickets	yes	yes	
T1558.001	Golden Ticket	yes	yes	
T1558.002	Silver Ticket	yes	yes	
T1558.003	Kerberoasting	yes	yes	
T1558.004	AS-REP Roasting	yes	yes	
T1559	Inter-Process Communication	yes	—	
T1559.001	Component Object Model	yes	yes	
T1559.002	Dynamic Data Exchange	yes	yes	
T1560	Archive Collected Data	yes	yes	
T1560.001	Archive via Utility	yes	yes	
T1560.002	Archive via Library	yes	yes	
T1560.003	Archive via Custom Method	yes	yes	
T1561	Disk Wipe	yes	yes	
T1561.001	Disk Content Wipe	yes	yes	
T1561.002	Disk Structure Wipe	yes	yes	
T1563	Remote Service Session Hijacking	yes	yes	
T1563.002	RDP Hijacking	yes	yes	
T1564	Hide Artifacts	yes	yes	
T1564.001	Hidden Files and Directories	yes	yes	
T1564.002	Hidden Users	yes	yes	
T1564.003	Hidden Window	yes	yes	
T1564.004	NTFS File Attributes	yes	yes	ETW
T1564.005	Hidden File System	yes	yes	ETW
T1564.006	Run Virtual Instance	yes	yes	
T1564.007	VBA Stomping	yes	yes	
T1564.008	Email Hiding Rules	yes	yes	cloud
T1564.010	Process Argument Spoofing	yes	yes	
T1564.011	Ignore Process Interrupts	yes	yes	
T1564.012	File/Path Exclusions	yes	yes	
T1565	Data Manipulation	yes	yes	
T1565.001	Stored Data Manipulation	yes	yes	
T1565.002	Transmitted Data Manipulation	yes	yes	
T1565.003	Runtime Data Manipulation	yes	yes	
T1566	Phishing	yes	yes	cloud
T1566.001	Spearphishing Attachment	yes	yes	cloud
T1566.002	Spearphishing Link	yes	yes	cloud
T1566.003	Spearphishing via Service	yes	yes	
T1566.004	Spearphishing Voice	—	yes	cloud
T1567	Exfiltration Over Web Service	yes	yes	

ID	Technique	Logs	Disk	Sources not taken
T1567.001	Exfiltration to Code Repository	yes	yes	
T1567.002	Exfiltration to Cloud Storage	yes	yes	
T1567.003	Exfiltration to Text Storage Sites	yes	yes	
T1567.004	Exfiltration Over Webhook	yes	yes	
T1568	Dynamic Resolution	yes	yes	
T1568.001	Fast Flux DNS	yes	yes	
T1568.002	Domain Generation Algorithms	yes	yes	
T1568.003	DNS Calculation	yes	yes	
T1569	System Services	yes	yes	
T1569.002	Service Execution	yes	yes	
T1570	Lateral Tool Transfer	yes	yes	
T1571	Non-Standard Port	yes	yes	
T1572	Protocol Tunneling	yes	yes	
T1573	Encrypted Channel	yes	yes	
T1573.001	Symmetric Cryptography	yes	yes	
T1573.002	Asymmetric Cryptography	yes	yes	
T1574	Hijack Execution Flow	yes	yes	
T1574.001	DLL	yes	yes	
T1574.005	Executable Installer File Permissions Weakness	yes	yes	
T1574.007	Path Interception by PATH Environment Variable	yes	yes	
T1574.008	Path Interception by Search Order Hijacking	yes	yes	
T1574.009	Path Interception by Unquoted Path	yes	yes	
T1574.010	Services File Permissions Weakness	yes	yes	
T1574.011	Services Registry Permissions Weakness	yes	yes	
T1574.012	COR_PROFILER	yes	yes	
T1574.013	KernelCallbackTable	yes	yes	ETW
T1574.014	AppDomainManager	yes	yes	
T1606	Forge Web Credentials	yes	yes	
T1606.001	Web Cookies	yes	yes	
T1606.002	SAML Tokens	yes	yes	4.0
T1611	Escape to Host	yes	yes	
T1614	System Location Discovery	yes	yes	ETW
T1614.001	System Language Discovery	yes	yes	ETW
T1615	Group Policy Discovery	yes	yes	NSM
T1620	Reflective Code Loading	yes	yes	ETW
T1621	Multi-Factor Authentication Request Generation	yes	yes	
T1622	Debugger Evasion	yes	yes	ETW

ID	Technique	Logs	Disk	Sources not taken
T1649	Steal or Forge Authentication Certificates	yes	—	
T1652	Device Driver Discovery	yes	yes	
T1653	Power Settings	yes	yes	
T1654	Log Enumeration	yes	yes	
T1657	Financial Theft	yes	yes	
T1659	Content Injection	yes	yes	NSM
T1665	Hide Infrastructure	yes	yes	NSM
T1667	Email Bombing	yes	yes	cloud
T1668	Exclusive Control	yes	yes	
T1669	Wi-Fi Networks	yes	yes	
T1673	Virtual Machine Discovery	yes	yes	
T1674	Input Injection	yes	yes	
T1678	Delay Execution	yes	yes	
T1679	Selective Exclusion	yes	yes	
T1680	Local Storage Discovery	yes	yes	
T1684	Social Engineering	yes	yes	
T1684.001	Impersonation	yes	yes	cloud
T1684.002	Email Spoofing	—	yes	cloud
T1685	Disable or Modify Tools	yes	yes	
T1685.001	Disable or Modify Windows Event Log	yes	yes	
T1685.003	Modify or Spoof Tool UI	yes	yes	
T1685.005	Clear Windows Event Logs	yes	yes	
T1685.007	Impair Autonomous Defenses*	—	—	AI
T1686	Disable or Modify System Firewall	yes	yes	
T1686.003	Windows Host Firewall	yes	yes	
T1687	Exploitation for Defense Impairment	yes	yes	
T1688	Safe Mode Boot	yes	yes	
T1689	Downgrade Attack	yes	yes	
T1690	Prevent Command History Logging	yes	yes	
T1698	Indirect Prompt Injection*	—	—	AI
T1699	Direct Prompt Injection*	—	—	AI
T1700	AI-Enabled Exfiltration	yes	yes	

ATT&CK Enterprise 20.0 (2026-10-13, SHA-256 81113BA98AFA2928…): 479 Windows techniques and sub-techniques; 475 with at least one trace WAC collects, 4 whose sources are all off the disk; 6 (marked *) have no Windows analytic in ATT&CK and are decided from the technique itself. *Logs*: a log WAC takes holds a trace of the technique; *Disk*: an artefact on the disk does, beyond the logs. A Windows event log is only there if the machine kept it: WAC records the channels' configuration and the audit policy, which say so.

Sources not taken, and why: ETW, live telemetry (ETW, API calls, performance counters): read by a

consumer as it happens; the trace files Windows writes to its disk are taken raw (etw_logs); NSM, network capture: on the wire, not on the machine; cloud, cloud service: in the tenant's logs; the machine's side is taken (cloud_identity, onedrive, intune_logs); EDR, an EDR product's own telemetry: in its console; Defender for Endpoint's local folder is taken raw (defender_endpoint); firmware, firmware: not read, WAC reads the disk; 4.0, server role (IIS, AD FS, domain controller): version 4.0; AI, an AI model's behaviour: ATT&CK names no trace of it on Windows

7.8 Source expected but missing

Conversely, some sources are on every installation of a given version, and Windows goes on without them: the `hosts` file, `Amcache.hve` and `SRUDB.dat` since they exist, the event logs, the services key. The table above marks them "always there from". When the collection does not hold such a source, it is neither a failed reading nor an absence of activity: it is a finding, and the artefact's file states it.

```
{
  "Artifact": "Hosts file",
  "CollectionStatus": "ExpectedButMissing",
  "WindowsRelease": "Windows 11",
  "WindowsBuild": 26200,
  "ExpectedLocation": "Windows\\System32\\drivers\\etc\\hosts",
  "Error": "0x80070002 The system cannot find the file specified.",
  "Note": "This version of Windows always has this artefact's source
           (artefacts.hosts) and the collection does not hold it. [...]"
}
```

The conversion says so on the console and records it in `conversion.json`. What it knows is that the sealed collection does not hold the source; that the machine did not have it reads in `investigation.json`, where the extraction of the folder is recorded with its result and the number of files taken. A file erased, a service switched off or a damaged system then remains to be explained.

> **Warning**
> The status is set only where the presence is certain. It is not for what exists only after an event (the `USBSTOR` key before the first USB storage), for what depends on an installed program (browsers, mail), nor on server editions, still to be measured: an absence there stays an artefact not collected, or an empty list.

7.9 Undecoded data

An item WAC cannot decode (unknown shell item, unrecognised extension block) comes with its raw bytes in hexadecimal, so that an analyst can decode it later. With `--dump`, the raw value of shellbags and shortcuts is always added.

7.10 Unsupported format

The format of a piece is read on the piece itself — its signature or its internal version —, never on the version of Windows: a Vista thumbnail cache found on a Windows 10 remains a Vista cache. When WAC has no decoder for the format met, be it an old one or one never seen yet, it converts nothing and says so; the piece itself is in the exhibit store, ready for another tool.


```
{
  "SID": "S-1-5-21-1004336348-1177238915-682003330-1000",
  "SIDName": "jean",
  "Cache": "C:\\Users\\jean\\AppData\\Local\\Microsoft\\Windows\\Explorer\\thumbcache_96.
    db",
  "FormatVersion": 20,
  "DecodingStatus": "UnsupportedFormat",
  "FormatSignature": "0x00000014",
  "FormatName": "CMMM, Vista layout",
  "Note": "Collected raw, not converted: this version of WAC has no
           decoder for this format. The exhibit store holds the source."
}
```


`FormatName` names the format when the table of section 7.6 knows it (column “Converted” at “no”); it is absent for an unknown format. The status is on a piece (`DecodingStatus`) when the artefact has several — Prefetch files, `$I` indexes of the recycle bin, thumbnail caches, jump lists (whose shortcuts are still decoded, without their DestList entry) — and on the whole artefact (`CollectionStatus`) when its source is single, like the ShimCache value.

Some formats carry no signature: the examined version of Windows then tells the form of the artefact. Before Windows 10 1607, `Amcache.hve` lacks its `Inventory` keys; before Windows 10 1709, BITS keeps its jobs in `qmgr0.dat`; before Windows 11, the search index is an ESE database. The collection takes the piece as usual, and the conversion says so instead of giving an empty list:


```
{
  "Artifact": "Amcache applications",
  "CollectionStatus": "UnsupportedFormat",
  "WindowsRelease": "Windows 8.1",
  "WindowsBuild": 9600,
  "FormatName": "registry hive, File and Programs keys",
  "Location": "Windows\\AppCompat\\Programs\\Amcache.hve",
  "Note": "On this version of Windows this artefact (artefacts.amcache) is
           in a format this version of WAC has no decoder for. [...]"
}
```


> **Warning**
> An “UnsupportedFormat” piece is neither an empty list nor a failed reading: the trace exists, it is collected, and its content remains to be read.

7.11 Paths

Paths and values are delivered **as they are on the disk**, untransformed: escaping happens only when the JSON is written.

CHAPTER 8

Traces left on the examined machine

Read this before using WAC on a case. This chapter lists what WAC does to the machine, operation by operation, including what it cannot avoid.

8.1 What WAC still does

Operation	Trace
Direct volume reading	One access per volume actually read (usually one). No file opened, so no access date modified. If object-access auditing is enabled, opening the volume may be logged (Security 4656/4663).
Writing the results	On the collection medium only. Nothing is written to the examined disk.
Replaying hive logs	On the working copy only, with an undo journal.
Service state	One read-only query to the Service Control Manager. No service is opened or modified.
Processes	A kernel snapshot; no process is opened. Owners: one query to the Terminal Services service.
Sessions	One read-only query to LSASS.
Network state	Reads of the TCP/IP stack's tables in memory: no packet sent, no name resolved. `iphlpapi.dll` is loaded only when `artefacts.network` is on.
Boot time	One in-memory query to the kernel.
Event logs (`--events`)	Read through the same direct volume access: the logging service is not solicited.
Referenced files (`--binary`)	Read through the same direct access. No file opened.
Authenticity check (`--binary`)	Catalogs read through the same direct access; verification in memory. No service solicited, no access to the certificate store, no network access.

8.2 What WAC no longer does

Earlier versions created a Volume Shadow Copy, went through COM and WMI, queried the Task Scheduler and the logging service, opened every process and every service, read the profile list in the live registry and opened every referenced file to fingerprint it. All these operations, which left entries in the logs or changed access dates, have been removed.

8.3 What no tool can avoid

These traces are listed so that the analyst recognises WAC's footprint instead of taking it for the activity under study:

- **running `WAC.exe` is an execution**: it produces an Amcache entry, a ShimCache entry at shutdown and, if WAC is run from the machine's disk rather than from a key, a Prefetch file and a BAM value. WAC writes no event itself, but a monitoring in place records its execution: Security 4688 and 4689 if process-creation auditing is on, Sysmon (the process, its raw reads of the volume, the files it creates on the key), an EDR. Expect to find WAC in the very artefacts it collects;
- **plugging in the USB stick** registers it in `USBSTOR`, `MountedDevices` and the related logs;
- **logging on** to run the tool creates a session and its log entries;
- WAC **takes up memory** during the collection, which may push pages out to the page file. Its reading of the volume bypasses the cache (unbuffered reads): the gigabytes read do not replace the machine's file cache.

CHAPTER 9

Known limitations

Volumes neither NTFS nor FAT
A file on a CD-ROM is not read: it is reported as unread.

Per-user variables
A path written with `%APPDATA%`, `%LOCALAPPDATA%` or `%USERPROFILE%` (in a scheduled task, for instance) is not resolved: the account concerned is not known there, and guessing would fingerprint another file.

Deleted executable
A process whose executable has been deleted from the disk has no fingerprint: only memory is left, outside WAC's scope.

Event messages
On the test machine, 99.6 % of the events get a readable message, identical to Windows' for 98 % of the kinds of event. The rest is explained in the validation file: Windows' own deviations from the manifest, language satellites older than their binary. The data themselves are always complete.

Signed but not authenticated
A few kinds of genuinely signed files are still collected, the cautious side: Microsoft Store applications (signed at package level, not file by file) and Windows Script Host scripts signed only within their text.

CHAPTER 10

Troubleshooting

Message or finding	Cause and remedy
“Check administrator rights” fails	The prompt was not started as administrator. Start it again with administrator rights.
“older than Windows 10” under the examined version of Windows	The collection is made, and every format WAC does not decode on that version is declared (section 7.10); the whole test cycle only covers Windows 10 and 11 so far.
“Checking the collection medium” fails, directory not empty	The output folder holds an earlier collection. Give a new folder with `--output`.
“Checking the collection medium” fails, not enough space	The medium is too small for the chosen options (section 3.2). Use a larger medium or drop `--binary`.
An artefact file holds `"CollectionStatus": "Not Collected"`	The source could not be read; the reason is in `Error`. The other artefacts are not affected.
Failed exhibits in the manifest	Referenced files missing from the disk (uninstalled software, service profile without a hive), non-NTFS volume… The reason is in `Result`.
The log reports binaries “hashed without being copied”	Space ran out during collection: the fingerprints are complete, the copies are not.
Need to understand a read failure	Run again with `--loglevel=2` (and, for NTFS reading, `--debug`) and read `WAC.exe.log`.

APPENDIX A

Glossary

Artefact
Trace left by the activity of the system or of a user, usable in an investigation.

Exhibit store (`exhibits`)
Directory of the raw copies, sealed, never rewritten.

Fingerprint
Value computed on a file's content (MD5, SHA-1, SHA-256): different contents have different fingerprints.

Hive
File holding part of the registry (`SYSTEM`, `SOFTWARE`, `ntuser.dat`…).

Manifest
File identifying each exhibit of the exhibit store.

`$MFT`
File table of an NTFS volume.

Seal
The manifest's fingerprint, kept apart: it reveals any change to the manifest.

Transaction log
`.LOG1`/`.LOG2` file holding a hive's recent changes, not yet written into it.

Working copy (`working`)
Directory of the working copies, on which the analysis and any modification are made.

WOF / Compact OS
Compressed storage mode of the system files of Windows 10 and 11.